

1	Reminder	1
1.1	Observations and Tricks	1
1.2	Bug List	1
2	Init (Linux)	1
2.1	vimrc	1
2.2	bashrc	2
3	Basic	2
3.1	Template (Using Codebook)	2
3.2	PBDS, Random	2
3.3	Debug	2
3.4	SVG Writer	2
3.5	Python	2
3.6	Stress Tests	2
4	Data Structure	3
4.1	Mo's Algorithm	3
4.2	CDQ	3
4.3	Persistent Treap	3
4.4	Li Chao Tree	3
4.5	Time Segment Tree	4
5	DP	4
5.1	SOS DP	4
6	Graph	4
6.1	Max Clique	4
6.2	Bellman-Ford	4
6.3	System of Difference Constraints	5
6.4	Graph Girth	5
6.5	Euler Trail	5
6.6	Vertex BCC (Round Square Tree)	5
6.7	Edge BCC	6
6.8	Kth Shortest Path	6
6.9	SCC - Tarjan	7
6.10	2SAT	7
7	Tree	7
7.1	Tree Isomorphism (Rooted Trees)	7
7.2	Tree Isomorphism (Unrooted Trees)	7
7.3	Heavy Light Decomposition	7
8	Matching	7
8.1	Bipartite Matching	7
8.2	Bipartite Weighted Matching	8
8.3	General Matching	8
8.4	General Weighted Matching	8
9	Flow	9
9.1	Flow Methods	9
9.2	Dinic	10
9.3	ISAP	11
9.4	Bounded Max Flow	11
9.5	MCMF	11
9.6	Push-Relabel	12
9.7	Gomory-Hu Tree	12
9.8	Global Min Cut	12
10	String	12
10.1	Rolling Hash	12
10.2	KMP	13
10.3	Z Value	13
10.4	Manacher	13
10.5	Suffix Array	13
10.6	Suffix Automaton	13
10.7	SA-IS	14
10.8	Minimum Rotation	14
10.9	Aho Corasick	14
11	Geometry	14
11.1	Template	14
11.2	Basic Operations	15
11.3	Lower Concave Hull	15
11.4	Pick's Theorem	15
11.5	Minimum Enclosing Circle	15
11.6	PolyUnion	15
11.7	Minkowski Sum	16
12	Number Theory	16
12.1	Prime Sieve and Defactor	16
12.2	Harmonic Series	17
12.3	Count Number of Divisors	17
12.4	數論分塊	17
12.5	Pollard's rho	17
12.6	Miller Rabin	17
12.7	Discrete Log	17
12.8	Discrete Sqrt	18
12.9	Fast Power	18
12.10	Extend GCD	18
12.11	Mu + Phi	18
12.12	Other Formulas	18
12.13	Polynomial	19
12.14	Counting Primes	20
12.15	Linear Sieve for Other Number Theoretic Functions	20
13	Linear Algebra	20
13.1	Gaussian-Jordan Elimination	20
13.2	Determinant	21
14	Combinatorics	21
14.1	Catalan Number	21
14.2	Bertrand's Ballot Theorem	21
14.3	Burnside's Lemma	21
15	Special Numbers	21
15.1	Fibonacci Series	21
15.2	Prime Numbers	21

1 Reminder

1.1 Observations and Tricks

- Contribution Technique
- 二分圖/Spanning Tree/DFS Tree
- 行、列操作互相獨立
- 奇偶性
- 當 s, t 遞增並且 $t = f(s)$, 對 s 二分搜不好做, 可以改成對 t 二分搜, 再算 $f(t)$
- 啟發式合併
- Permutation Normalization (做一些平移對齊兩個 permutation)
- 枚舉 $a_1 \sim a_n$ 再枚舉 $a_n \sim a_1$ 可以包在一個迴圈
- 兩個凸型函數相加還是凸型函數, 相減不一定
- 一個區間的 $\text{mex} = k$, 表示這個區間包含 $0 \sim k-1$ 所有數字, 並且「 U - 區間」的最小值 $= k$ 。

1.2 Bug List

- 沒開 long long
- 陣列截出界／陣列開不夠大
- 寫好的函式忘記呼叫
- 0-base / 1-base
- 忘記初始化
- == 打成 =
- <= 打成 <+
- dp[i] 從 dp[i-1] 轉移時忘記特判 $i > 0$
- std::sort 比較運算子寫成 < 或是讓 = 的情況為 true
- 漏 case
- 線段樹改值懶標初始值不能設為 0
- DFS 的時候不小心覆寫到全域變數
- 浮點數誤差
- unsigned int128
- 多筆測資不能沒讀完直接 return
- 記得刪 cerr
- vector 超級肥，小 vector 請用 array，例如矩陣快速冪

This image shows a full page of blank graph paper. The grid consists of thin, light gray horizontal and vertical lines that intersect to form a uniform pattern of small squares across the entire page. There are no margins, text, or other markings present.

2 Init (Linux)

2.1 vimrc

```

1 syn on
2 set nu rnu cul mouse=a cin et ts=2 sw=2 sts=2 autochdir
  cb=unnamedplus
3 no ; :
4 ino {<CR> {<CR>}<Esc>ko
5 ca Hash w !cpp -dD -P \ | tr -d '[:space:]' \ | md5sum \ | cut
  -c-6

1 " Gino's .vimrc "
2 "no <C-h> ^ | <C-l> $ | h b | l e | b l | e h

```

2.2 bashrc

```
1 gogo() {
2   g++ -std=c++20 -O2 "$1" && echo run && ./a.out
3 }
4 alias vig='vim -u ~/.vimrc-gino'
```

3 Basic

3.1 Template (Using Codebook)

```
1 // CRYptoGRapHer
2 #define SZ(c) ((int)(c).size())
3 // kactl
4 #define sz(x) (int)(x).size()
5 #define rep(i,a,b) for(int i=(a);i<(b);i++)
6 using ll = long long;
7 using vi = vector<int>;
8 using pii = pair<int, int>;
9 // ckiseki
10 #define all(x) begin(x), end(x)
11 #define pb emplace_back
12 #define REP(i, l, r) for (int i=(l); i<=(r); ++i)
13 const ll LINF = 1e18;
14 template<class A, class B> bool chmin(A& a, B& b) {
15     return b < a ? a = b, 1 : 0;
16 }
```

3.2 PBDS, Random

```
1 #include <bits/extc++.h>
2 using namespace __gnu_pbds;
3 // map
4 tree<int, int, less<>, rb_tree_tag,
5   tree_order_statistics_node_update> tr;
6 tr.order_of_key(element);
7 tr.find_by_order(rank);
8 // set
9 tree<int, null_type, less<>, rb_tree_tag,
10   tree_order_statistics_node_update> tr;
11 tr.order_of_key(element);
12 tr.find_by_order(rank);
13 // priority queue
14 __gnu_pbds::priority_queue<int, less<int>> big_q;
15 __gnu_pbds::priority_queue<int, greater<int>> small_q;
16 ql.join(q2); // join
17 mt19937
18 gen(chrono::steady_clock::now().time_since_epoch().count());
19 uniform_int_distribution<int>(a, b)(rng) // [a, b]
20 uniform_real_distribution<double>(a, b)(rng) // (a, b)
21 shuffle(v.begin(), v.end(), gen);
```

3.3 Debug

```
1 #define debug(...) cerr << "\x1b[33m" #__VA_ARGS__ " : \
2   \x1b[0m" \
3   [ (auto&&... a) { (cerr << a << ' ', ...); }
4   (__VA_ARGS__), cerr << '\n'
5 #define output(...) \
6   [ (auto&&... a) { (cout << a << ' ', ...); }
7   (__VA_ARGS__), cout << '\n'
8 int v = 42;
9 format("{:b}", v); // "101010"
10 format("{:#b}", v); // "0b101010"
11 format("{:x}", v); // "0x2a"
12 format("{:X}", v); // "0X2A"
13 int x = 7;
14 format("{:05d}", x); // "00007"
15 format("{:>5}", x); // " 7"
16 format("{:<5}", x); // "7 "
17 format("{:^5}", x); // " 7 "
18 format("{:*5}", x); // "***7***"
19 double pi = 3.1415926535;
20 format("{:.2f}", pi); // "3.14"
```

3.4 SVG Writer

```
1 // Author: ckiseki
2 class SVG {
3     void p(string_view s) { o << s; }
4     void p(string_view s, auto v, auto... vs) {
5         auto i = s.find('$');
6         o << s.substr(0, i) << v, p(s.substr(i + 1), vs...);
7     }
8     ostream o; string c = "red";
9 public:
10     SVG(auto f, auto x1, auto y1, auto x2, auto y2) : o(f) {
11         p("<svg xmlns='http://www.w3.org/2000/svg' "
12           "viewBox='$ $ $ $'>\n"
13           "<style*>{stroke-width:0.5%;}</style>\n",
14           x1, -y2, x2 - x1, y2 - y1); }
15     ~SVG() { p("</svg>\n"); }
16     void color(string nc) { c = nc; }
17     void line(auto x1, auto y1, auto x2, auto y2) {
```

```
18     p("<line x1='$' y1='$' x2='$' y2='$' stroke='$'>\n",
19       x1, -y1, x2, -y2, c); }
20     void circle(auto x, auto y, auto r) {
21         p("<circle cx='$' cy='$' r='$' stroke='$' "
22           "fill='none'/>\n", x, -y, r, c); }
23     void text(auto x, auto y, string s, int w = 12) {
24         p("<text x='$' y='$' font-size='$px'>$</text>\n",
25           x, -y, w, s); }
26 };
```

```
1 SVG svg("out.svg", -200, -200, 200, 200);
2 svg.color("lightgray"); // drawing x-y plane
3 svg.line(-200, 0, 200, 0); svg.line(0, -200, 0, 200);
4 svg.color("blue"); svg.line(10, 10, 80, 80);
5 svg.color("red"); svg.circle(50, 50, 1);
```

3.5 Python

```
1 import sys
2 input = sys.stdin.readline
3 readStr = lambda: input().rstrip('\r\n')
4 readInt = lambda: int(input())
5 readInts = lambda: list(map(int, input().split()))
6
7 from decimal import *
8 getcontext().prec = 50 # precision
9 x = Decimal(str(x))
10 y = Decimal("6.7")
11 x *= y
12 print(x.quantize(Decimal("0.000001"),
13   rounding=ROUND_HALF_EVEN))
14 # ROUND_CEILING : To INF (2.9 -> 3, -2.1 -> -2)
15 # ROUND_FLOOR : To -INF (2.1 -> 2, -2.9 -> -3)
16 # ROUND_HALF_EVEN: To nearest even (2.5 -> 2, 3.5 -> 4)
17 # ROUND_HALF_UP : Half away from 0 (2.5 -> 3, -2.5 -> -3)
18 # ROUND_HALF_DOWN: Half towards 0 (2.5 -> 2, -2.5 -> -2)
19 # ROUND_UP : Away from 0 (2.1 -> 3, -2.1 -> -3)
20 # ROUND_DOWN : Truncate to 0 (2.9 -> 2, -2.9 -> -2)
21 # <Default>: ROUND_HALF_EVEN
22 # <Standard School Math>: ROUND_HALF_UP
```

3.6 Stress Tests

```
1 from random import *
2 n = randint(1, 10)
3 arr = list(randint(1, 20) for i in range(n))
4 print(*arr)
5 # permutation / non-repetitive sequence
6 perm = sample(range(1, n + 1), n)
7 arr = sample(range(1, 10000 + 1), n)
8 # string
9 from string import *
10 s = "".join(choices(ascii_lowercase, k=n))
11 t = "".join(choices(ascii_lowercase[:3], k=n)) # (a|b|c)*
12 u = "".join(choices(ascii_lowercase + ascii_uppercase +
13   digits, k=n))
14 # palindrome
15 half = "".join(choices(ascii_lowercase, k=(n + 1) // 2))
16 palindrome = half + (half[-2::-1] if n % 2 else half[::-1])
17 # ===== tree =====
18 n = 30
19 ## shallow tree
20 edges = [(randint(1, u - 1), u) for u in range(2, n + 1)]
21 ## deep tree
22 (suggested k: 2 ~ 5 or int(sqrt(n)) or 2n/D (D is
23   expected depth))
24 from math import *
25 k = int(sqrt(n))
26 edges = [(randint(max(1, u - k), u - 1), u) for u in range(2, n
27   + 1)]
28 for e in edges:
29     print(*e)
30
31 edges = [(1, i) for i in range(2, n + 1)] ## star
32 edges = [(i, i + 1) for i in range(1, n)] ## chain
33 edges = [(i // 2, i) for i in range(2, n + 1)] ## complete
34   binary tree
35 # ===== graph =====
36 # not necessarily connected, no multi-edge or self-loop
37 n = 10
38 density = 0.3 # graph density (0.0 ~ 1.0)
39 edges = [(i, j) for i in range(1, n+1) for j in range(i+1,
40   n+1) if random() < density]
41 ## connected, no multi-edge or self-loop
42 n, extra_edges = 10, 5
43 edges = set()
44 ##### ... generate spanning tree with codes above
45 while len(edges) < n - 1 + extra_edges:
46     u, v = randint(1, n), randint(1, n)
47     if u != v:
48         edges.add((min(u, v), max(u, v)))
49 edges = list(edges)
```

```

45 ## DAG
46 perm = sample(range(1, n + 1), n)
47 edges = []
48 for _ in range(m):
49     i, j = sorted(sample(range(n), 2))
50     edges.append((perm[i], perm[j]))
51 ## Bipartite Graph
52 n1, n2 = 6, 7
53 left, right = list(range(1, n1+1)), list(range(n1+1,
54 n1+n2+1))
55 edges = [(choice(left), choice(right)) for _ in range(m)]
56
57 g++ -std=c++20 -o buggy $1
58 g++ -std=c++20 -o ac ac.cpp
59 for i in {1..100}; do
60     echo "TEST $i"
61     python3 gen.py > in
62     ./ac < in > oa
63     ./buggy < in > ob
64     if ! diff -q oa ob; then
65         cat in <(echo "=== AC") oa <(echo "=== WA") ob
66         break
67     fi
68 done

```

4 Data Structure

4.1 Mo's Algorithm

```

1 // segments are 0-based
2 int B = max(1, (int)sqrt(n));
3 sort(Q.begin(), Q.end(), [&](const auto& a, const auto& b) {
4     if (a[0]/B != b[0]/B) return a[0]/B < b[0]/B;
5     return (a[0]/B & 1) ? a[1] < b[1] : a[1] > b[1];
6 });
7 ll cur = 0; // current answer
8 int pl = 0, pr = -1;
9 for (auto& qi : Q) {
10     // get (l, r, qid) from qi
11     while (pl < l) del(pl++); while (pl > l) add(--pl);
12     while (pr < r) add(++pr); while (pr > r) del(pr--);
13     ans[qid] = cur;
14 }

```

4.2 CDQ

```

1 // preparation
2 // 1. write a 1-base BIT
3 // 2. init(n, mxc): n is number of elements, mxc = 值域
4 // 3. build info array CDQ.v by yourself (x, y, z, id)
5 // 4. call solve()
6 // => cdq.ans[i] is number of j s.t. (xj <= xi, yj <= yi, zj
7 // <= zi) (j != i)
8 struct CDQ {
9     struct info {
10         int x, y, z, id;
11         info(int x, int y, int z, int id):
12             x(x), y(y), z(z), id(id) {}
13         info():
14             x(0), y(0), z(0), id(0) {}
15     };
16     int n, mxc;
17     BIT bit;
18     vector<info> v;
19     vector<ll> ans;
20     void init(int n, int _mxc) {
21         n = _n; mxc = _mxc;
22         v.assign(n, info{});
23         ans.assign(n, 0);
24     }
25     void dfs(int l, int r) {
26         if (l >= r) return;
27         int mid = (l+r) >> 1;
28         dfs(l, mid); dfs(mid+1, r);
29
30         vector<info> tmp;
31         vector<int> bit_op;
32         int pl = l, pr = mid+1;
33
34         while (pl <= mid && pr <= r) {
35             if (v[pl].y <= v[pr].y) {
36                 tmp.emplace_back(v[pl]);
37                 bit.upd(v[pl].z, 1);
38                 bit_op.emplace_back(v[pl].z);
39                 pl++;
40             }
41             else {
42                 tmp.emplace_back(v[pr]);
43                 ans[v[pr].id] += bit.qry(v[pr].z);
44                 pr++;
45             }
46         }
47         while (pl <= mid) {
48             tmp.emplace_back(v[pl]);
49             bit.upd(v[pl].z, 1);
50             bit_op.emplace_back(v[pl].z);
51             pl++;
52         }
53         for (auto& p : tmp) {
54             ans[p.id] -= bit.qry(p.z);
55             bit.upd(p.z, -1);
56         }
57     }
58     void solve() {
59         dfs(0, n-1);
60     }
61 };

```

```

46 while (pl <= mid) {
47     tmp.emplace_back(v[pl]);
48     pl++;
49 }
50 while (pr <= r) {
51     tmp.emplace_back(v[pr]);
52     ans[v[pr].id] += bit.qry(v[pr].z);
53     pr++;
54 }
55 for (int i = l, j = 0; i <= r; i++, j++) v[i] = tmp[j];
56 for (auto& op : bit_op) bit.upd(op, -1);
57 }
58 void solve() {
59     bit.init(mxc);
60     sort(v.begin(), v.end(), [&](const info& a, const info&
61 b){
62     return (a.x == b.x ? (a.y == b.y ? (a.z == b.z ?
63 a.id < b.id : a.z < b.z) : a.y < b.y) : a.x < b.x);
64 });
65 dfs(0, n-1);
66 map<pair<int, pii>, int> s;
67 sort(v.begin(), v.end(), [&](const info& a, const info&
68 b){
69     return a.id > b.id;
70 });
71 for (int i = 0, id = n-1; i < n; i++, id--) {
72     pair<int, pii> tmp = make_pair(v[i].x,
73 make_pair(v[i].y, v[i].z));
74     auto it = s.find(tmp);
75     if (it != s.end())
76         ans[id] += it->second;
77     s[tmp]++;
78 }
79 }

```

4.3 Persistent Treap

```

1 // Author: Ian
2 struct node {
3     node *l, *r;
4     char c; int v, sz;
5     node(char x = '$'): c(x), v(mt()), sz(1) {
6         l = r = nullptr;
7     }
8     node(node* p) { *this = *p; }
9     void pull() {
10         sz = 1;
11         for (auto i : {l, r})
12             if (i) sz += i->sz;
13     }
14     arr[maxn], *ptr = arr;
15     inline int size(node* p) { return p ? p->sz : 0; }
16     node* merge(node* a, node* b) {
17         if (!a || !b) return a ? a : b;
18         if (a->v < b->v) {
19             node* ret = new(ptr++) node(a);
20             ret->r = merge(ret->r, b); ret->pull();
21             return ret;
22         }
23         else {
24             node* ret = new(ptr++) node(b);
25             ret->l = merge(a, ret->l); ret->pull();
26             return ret;
27         }
28     }
29     P<node*> split(node* p, int k) {
30         if (!p) return {nullptr, nullptr};
31         if (k >= size(p->l) + 1) {
32             auto [a, b] = split(p->r, k - size(p->l) - 1);
33             node* ret = new(ptr++) node(p);
34             ret->r = a, ret->pull();
35             return {ret, b};
36         }
37         else {
38             auto [a, b] = split(p->l, k);
39             node* ret = new(ptr++) node(p);
40             ret->l = b, ret->pull();
41             return {a, ret};
42         }
43     }

```

4.4 Li Chao Tree

```

1 // Author: Ian
2 // Function: For a set of lines L, find the maximum L_i(x)
3 // in L in O(lg n).
4 typedef long double ld;
5 constexpr int maxn = 5e4 + 5;
6 struct line {
7     ld a, b;

```

```

7 ld operator()(ld x) {return a * x + b;}
8 } arr[(maxn + 1) << 2];
9 bool operator<(line a, line b) {return a.a < b.a;}
10 #define m ((l+r)>>1)
11 void insert(line x, int i = 1, int l = 0, int r = maxn) {
12     if (r - l == 1) {
13         if (x(l) > arr[i](l))
14             arr[i] = x;
15         return;
16     }
17     line a = max(arr[i], x), b = min(arr[i], x);
18     if (a(m) > b(m))
19         arr[i] = a, insert(b, i << 1, l, m);
20     else
21         arr[i] = b, insert(a, i << 1 | 1, m, r);
22 }
23 ld query(int x, int i = 1, int l = 0, int r = maxn) {
24     if (x < l || r <= x) return -numeric_limits<ld>::max();
25     if (r - l == 1) return arr[i](x);
26     return max({arr[i](x), query(x, i << 1, l, m), query(x, i
    << 1 | 1, m, r)});
27 }
28 #undef m

```

4.5 Time Segment Tree

```

1 // Author: Ian
2 constexpr int maxn = 1e5 + 5;
3 V<P<int>> arr[(maxn + 1) << 2];
4 V<int> dsu, sz;
5 V<tuple<int, int, int>> his;
6 int cnt, q;
7 int find(int x) {
8     return x == dsu[x] ? x : find(dsu[x]);
9 };
10 inline bool merge(int x, int y) {
11     int a = find(x), b = find(y);
12     if (a == b) return false;
13     if (sz[a] > sz[b]) swap(a, b);
14     his.emplace_back(a, b, sz[b]), dsu[a] = b, sz[b] +=
        sz[a];
15     return true;
16 };
17 inline void undo() {
18     auto [a, b, s] = his.back(); his.pop_back();
19     dsu[a] = a, sz[b] = s;
20 }
21 #define m ((l + r) >> 1)
22 void insert(int ql, int qr, P<int> x, int i = 1, int l = 0,
    int r = q) {
23     // debug(ql, qr, x); return;
24     if (qr <= l || r <= ql) return;
25     if (ql <= l && r <= qr) {arr[i].push_back(x); return;}
26     if (qr <= m)
27         insert(ql, qr, x, i << 1, l, m);
28     else if (m <= ql)
29         insert(ql, qr, x, i << 1 | 1, m, r);
30     else {
31         insert(ql, qr, x, i << 1, l, m);
32         insert(ql, qr, x, i << 1 | 1, m, r);
33     }
34 }
35 void traversal(V<int>& ans, int i = 1, int l = 0, int r = q)
    {
36     int opcnt = 0;
37     // debug(i, l, r);
38     for (auto [a, b] : arr[i])
39         if (merge(a, b))
40             opcnt++, cnt--;
41     if (r - l == 1) ans[l] = cnt;
42     else {
43         traversal(ans, i << 1, l, m);
44         traversal(ans, i << 1 | 1, m, r);
45     }
46     while (opcnt--)
47         undo(), cnt++;
48     arr[i].clear();
49 }
50 #undef m
51 inline void solve() {
52     int n, m; cin >> n >> m >> q; q++;
53     dsu.resize(cnt = n), sz.assign(n, 1);
54     iota(dsu.begin(), dsu.end(), 0);
55     // a, b, time, operation
56     unordered_map<ll, V<int>> s;
57     for (int i = 0; i < m; i++) {
58         int a, b; cin >> a >> b;
59         if (a > b) swap(a, b);
60         s[(ll)a << 32 | b].emplace_back(0);
61     }

```

```

62     for (int i = 1; i < q; i++) {
63         int op, a, b;
64         cin >> op >> a >> b;
65         if (a > b) swap(a, b);
66         switch (op) {
67             case 1:
68                 s[(ll)a << 32 | b].push_back(i);
69                 break;
70             case 2:
71                 auto tmp = s[(ll)a << 32 | b].back();
72                 s[(ll)a << 32 | b].pop_back();
73                 insert(tmp, i, P<int> {a, b});
74             }
75     }
76     for (auto [p, v] : s) {
77         int a = p >> 32, b = p & -1;
78         while (v.size()) {
79             insert(v.back(), q, P<int> {a, b});
80             v.pop_back();
81         }
82     }
83     V<int> ans(q);
84     traversal(ans);
85     for (auto i : ans)
86         cout << i << ' ';
87     cout << endl;
88 }

```

5 DP

5.1 SOS DP

```

1 // Author: Gino
2 // Function: Solve problems that enumerates subsets of
    subsets (3^n => n*2^n)
3 for (int msk = 0; msk < (1<<n); msk++) {
4     for (int i = 1; i <= n; i++) {
5         if (msk & (1<<(i - 1))) {
6             dp[msk][i] = dp[msk][i - 1] + dp[msk ^ (1<<(i - 1))][i -
                1];
7         } else {
8             dp[msk][i] = dp[msk][i - 1];
9         }
10     }

```

6 Graph

6.1 Max Clique

```

1 // max N = 64
2 typedef unsigned long long ll;
3 struct MaxClique {
4     ll nb[N], n, ans;
5     void init(ll _n) {
6         n = _n;
7         for (int i = 0; i < n; i++) nb[i] = 0LLU;
8     }
9     void add_edge(ll _u, ll _v) {
10         nb[_u] |= (1LLU << _v);
11         nb[_v] |= (1LLU << _u);
12     }
13     void B(ll r, ll p, ll x, ll cnt, ll res) {
14         if (cnt + res < ans) return;
15         if (p == 0LLU && x == 0LLU) {
16             if (cnt > ans) ans = cnt;
17             return;
18         }
19         ll y = p | x; y &= -y;
20         ll q = p & (~nb[ int( log2( y ) ) ]);
21         while (q) {
22             ll i = int( log2( q & (-q) ) );
23             B( r | (1LLU << i), p & nb[i], x & nb[i]
                & nb[i] );
24             cnt + 1LLU, __builtin_popcountll( p &
                nb[i] );
25             q &= ~(1LLU << i);
26             p &= ~(1LLU << i);
27             x |= (1LLU << i);
28         }
29     }
30     int solve() {
31         ans = 0;
32         ll _set = 0;
33         if (n < 64) _set = (1LLU << n) - 1;
34         else {
35             for (ll i = 0; i < n; i++) _set |= (1LLU << i);
36         }
37         B( 0LLU, _set, 0LLU, 0LLU, n );
38         return ans;
39     }
40 } maxClique;

```

6.2 Bellman-Ford

```

1 // Author: Gino

```

```

2// n, m = #(vertices), #(edges), and vertices are 1-based
3// 1. BellmanFord bf(G, n, m);
4// 2-1: bf.calcDis(s);
5// => bf.dis[u] := shortest dis from s to u
6// => bf.dis[u] := LINF (s can't reach u)
7// => bf.dis[u] := -LINF (dis[u] can be arbitrary small)
8// 2-2: bf.findNegCycle();
9// => return false (if no neg cycle)
10// => return true (has neg cycle, and gives an example:
    bf.neg_cycle)
11// Time: O(VE), Space: O(V + E)
12const ll LINF = 4e18;
13struct BellmanFord {
14    const vector<vector<pair<int, ll>>>& G;
15    int n, m; // #(vertices), #(edges)
16    BellmanFord(const auto& G, int n, int m):
        G(G), n(n), m(m) {}
17
18    vector<ll> dis;
19    vector<int> nth_relax, pa;
20    void run_bf(const auto& src) {
21        dis.assign(n + 1, LINF);
22        nth_relax.assign(n + 1, 0);
23        pa.assign(n + 1, -1);
24        for (auto& s : src) dis[s] = 0;
25        for (int rlx = 1; rlx <= n; rlx++) {
26            for (int u = 1; u <= n; u++) {
27                if (dis[u] == LINF) continue; // !important
28                for (auto& [v, w] : G[u]) {
29                    if (dis[v] > dis[u] + w) {
30                        dis[v] = dis[u] + w;
31                        pa[v] = u;
32                        if (rlx == n) nth_relax[v] = 1;
33                    }
34                }
35            }
36            // 5 brackets
37            void calcDis(int s) {
38                run_bf(vector<int>{s});
39                queue<int> q;
40                for (int u = 1; u <= n; u++)
41                    if (nth_relax[u]) q.push(u);
42                while (!q.empty()) {
43                    int u = q.front(); q.pop();
44                    for (auto& [v, w] : G[u]) {
45                        if (!nth_relax[v]) {
46                            nth_relax[v] = 1;
47                            q.push(v);
48                        }
49                    }
50                }
51                vector<int> neg_cycle;
52                bool findNegCycle() {
53                    auto src = views::iota(1, n + 1);
54                    run_bf(src);
55                    auto it = ranges::find_if(src, [&](int s) { return
nth_relax[s]; });
56                    if (it == src.end()) return false;
57                    int ptr = *it;
58                    for (int i = 0; i < n; i++) ptr = pa[ptr];
59                    neg_cycle.clear();
60                    int cur = ptr;
61                    while (true) {
62                        neg_cycle.emplace_back(cur);
63                        if (cur == ptr && neg_cycle.size() > 1) break;
64                        cur = pa[cur];
65                    }
66                    ranges::reverse(neg_cycle);
67                    return true;
68                }
69            }
70        }
71    }

```

6.3 System of Difference Constraints

```

1vector<vector<pair<int, ll>>> G;
2void add(int u, int v, ll w) { G[u].push_back({v, w}); }

```

- $x_u - x_v \leq c \Rightarrow \text{add}(v, u, c)$
- $x_u - x_v \geq c \Rightarrow \text{add}(u, v, -c)$
- $x_u - x_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)$
- $x_u \geq c \Rightarrow \text{add super vertex } x_0 = 0, \text{ then } x_u - x_0 \geq c \Rightarrow \text{add}(u, 0, -c)$
- Don't forget non-negative constraints for every variable if specified implicitly.
- Interval sum $\Rightarrow$ Use prefix sum to transform into differential constraints. Don't forget $S_{i+1} - S_i \geq 0$ if x_i needs to be non-negative.
- $x_u/x_v \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

6.4 Graph Girth

Run BFS for every node, when encountered non-BFS-tree edge, update answer (min cycle length) with $\text{dis}[u] + \text{dis}[v] + 1$. Time $O(VE)$.

6.5 Euler Trail

```

1// Author: kactl (Simon Lindholm), wrapped by Gino
2// Vertices are 1-based, Edge ID are 0-based
3// 1. EulerWalk euler(n, true(directed) /
    false(undirected));
4// 2. euler.addEdge(u, v);
5// 3. int result = euler.solve();
6// => -1: nothing (euler.path, euler.path_edges = {})
7// => 1: Euler trail,
8// => 2: Euler circuit
9//     euler.path = {u_1, ..., u_{m+1}}
10//     euler.path_edges = {e_1, ..., e_m}
11// Time: O(V + E), Space: O(V + E)
12// Test: V, E <= 2e5, 99ms
13struct EulerWalk {
14    int n, m = 0; bool dir;
15    vector<vector<pii>> G; vi deg;
16    EulerWalk(int n, bool dir) : n(n), dir(dir), G(n + 1),
deg(n + 1) {}
17    void addEdge(int u, int v) {
18        G[u].push_back({v, m});
19        if (!dir) G[v].push_back({u, m});
20        deg[u]++, deg[v] += dir ? -1 : 1;
21        m++;
22    }
23    vi path, path_edges;
24    void walk(int src) {
25        path.clear(); path_edges.clear();
26        vi its(n + 1, visE(m));
27        function<void(int, int)> dfs = [&](int u, int e_in) {
28            while (its[u] < sz(G[u])) {
29                auto [v, e] = G[u][its[u]++];
30                if (visE[e]) continue;
31                visE[e] = 1;
32                dfs(v, e);
33            }
34            path.push_back(u);
35            if (e_in != -1) path_edges.push_back(e_in);
36        };
37        dfs(src, -1);
38        if (sz(path) != m + 1) {
39            path.clear(); path_edges.clear(); return;
40        }
41        ranges::reverse(path); ranges::reverse(path_edges);
42    }
43    int solve() {
44        int s = -1, odd = 0, src = 0, dst = 0;
45        for (int u = 1; u <= n; u++) {
46            if (!dir && (deg[u] & 1)) odd++, s = u;
47            if (dir && deg[u] == 1) s = u, src++;
48            if (dir && deg[u] == -1) dst++;
49            if (dir && abs(deg[u]) > 1) return -1;
50        }
51        if (!dir && odd != 0 && odd != 2) return -1;
52        if (dir && !(src == 0 && dst == 0) || (src == 1 && dst
== 1)) return -1;
53        if (s == -1) for (int u = 1; u <= n; u++) if (!
G[u].empty()) { s = u; break; }
54        if (s == -1) s = 1;
55        walk(s); return path.empty() ? -1 : path.front() ==
path.back() ? 2 : 1;
56    }
57};

```

6.6 Vertex BCC (Round Square Tree)

```

1// Author: std_abs
2struct BCC { // 1-based, remember to build
3    int n, nbcc; // note for isolated point
4    vector<vector<int>> g, _g; // id > n: bcc
5    vector<int> pa, dep, low, stk, pa2, dep2;
6    void dfs(int v, int p) {
7        dep[v] = low[v] = ~p ? dep[p] + 1 : 0;
8        stk.pb(v), pa[v] = p;
9        for (int u : g[v]) if (u != p) {
10            if (low[u] == -1) {
11                dfs(u, v), low[v] = min(low[v], low[u]);
12                if (low[u] >= dep[v]) {
13                    int id = nbcc++, x;
14                    do {
15                        x = stk.back(), stk.pop_back();
16                        _g[id + n + 1].pb(x), _g[x].pb(id + n + 1);
17                    } while (x != u);
18                    _g[id + n + 1].pb(v), _g[v].pb(id + n + 1);
19                }
20            }
21        }
22    }
23};

```



```

20     } else low[v] = min(low[v], dep[u]);
21 }
22 }
23 bool is_cut(int x) { return sz(_g[x]) != 1; }
24 vector<int> bcc(int id) { return _g[id + n + 1]; }
25 int bcc_id(int u, int v) {
26     return pa2[dep2[u] < dep2[v] ? v : u] - n - 1; }
27 void dfs2(int v, int p) {
28     dep2[v] = ~p ? dep2[p] + 1 : 0; pa2[v] = p;
29     for (int u : _g[v]) if (u != p) dfs2(u, v);
30 }
31 void build() {
32     low.assign(n + 1, -1);
33     for (int i = 1; i <= n; ++i) if (low[i] == -1)
34         dfs(i, -1), dfs2(i, -1);
35 }
36 void add_edge(int u, int v) {
37     g[u].pb(v), g[v].pb(u); }
38 BCC (int _n) : n(_n), nbcc(0), g(n + 1), _g(2 * n + 1),
39     pa(n + 1), dep(n + 1), low(n + 1), stk(), pa2(2 * n +
40     1), dep2(2 * n + 1) {}
41 };

```

6.7 Edge BCC

```

1 // Author: std_abs
2 // Vertices 1-based
3 // Edge ID 0-based, BCCID 0-based
4 // 1. EBCC ebcc(n); // n = #(vertices)
5 // 2. ebcc.add_edge(u, v);
6 // 3. ebcc.build();
7 // => nbcc: number of EBCC
8 // => bccs: stores all edge bccs
9 // => bcc_id[u], is_bridge[Edge ID]
10 struct EBCC { // 1-based, remember to build
11     int n, m, nbcc;
12     vector<vector<pair<int, int>>> G;
13     vector<vector<int>> bccs;
14     vector<int> pa, low, dep, bcc_id, stk, is_bridge;
15     void dfs(int v, int p, int f) {
16         low[v] = dep[v] = ~p ? dep[p] + 1 : 0;
17         stk.push_back(v), pa[v] = p;
18         for (auto [u, e] : G[v]) {
19             if (low[u] == -1)
20                 dfs(u, v, e), low[v] = min(low[v], low[u]);
21             else if (e != f)
22                 low[v] = min(low[v], dep[u]);
23         }
24         if (low[v] == dep[v]) {
25             if (~f) is_bridge[f] = true;
26             bccs.push_back({});
27             int id = nbcc++, x;
28             do {
29                 x = stk.back(), stk.pop_back();
30                 bcc_id[x] = id;
31                 bccs[id].emplace_back(x);
32             } while (x != v);
33         }
34     }
35     void build() {
36         is_bridge.assign(m, 0);
37         for (int i = 1; i <= n; ++i) if (low[i] == -1)
38             dfs(i, -1, -1);
39     }
40     void add_edge(int u, int v) {
41         G[u].emplace_back(v, m), G[v].emplace_back(u, m++);
42     }
43     EBCC (int _n) : n(_n), m(0), nbcc(0), G(n + 1), pa(n + 1),
44         low(n + 1, -1), dep(n + 1), bcc_id(n + 1), stk() {}
45 };

```

6.8 Kth Shortest Path

```

1 // time: O(|E| lg |E| + |V| lg |V| + K)
2 // memory: O(|E| lg |E| + |V|)
3 struct KSP { // 1-base
4     struct nd {
5         int u, v; ll d;
6         nd(int ui=0, int vi=0, ll di=INF) { u=ui; v=vi; d=di; }
7     };
8     struct heap { nd* edge; int dep; heap* chd[4]; };
9     static int cmp(heap* a, heap* b)
10     { return a->edge->d > b->edge->d; }
11     struct node {
12         int v; ll d; heap* H; nd* E;
13         node() {}
14         node(ll _d, int _v, nd* _E) { d=_d; v=_v; E=_E; }
15         node(heap* _H, ll _d) { H=_H; d=_d; }
16         friend bool operator<(node a, node b)
17         { return a.d > b.d; }

```

```

18     };
19     int n, k, s, t, dst[N]; nd *nxt[N];
20     vector<nd*> g[N], rg[N]; heap *nullNd, *head[N];
21     void init(int _n, int _k, int _s, int _t) {
22         n=_n; k=_k; s=_s; t=_t;
23         for (int i=1; i<=n; i++) {
24             g[i].clear(); rg[i].clear();
25             nxt[i]=NULL; head[i]=NULL; dst[i]=-1;
26         }
27     }
28     void addEdge(int ui, int vi, ll di) {
29         nd* e=new nd(ui, vi, di);
30         g[ui].push_back(e); rg[vi].push_back(e);
31     }
32     queue<int> dfsQ;
33     void dijkstra() {
34         while (dfsQ.size()) dfsQ.pop();
35         priority_queue<node> Q; Q.push(node(0, t, NULL));
36         while (!Q.empty()) {
37             node p=Q.top(); Q.pop(); if (dst[p.v] != -1) continue;
38             dst[p.v]=p.d; nxt[p.v]=p.E; dfsQ.push(p.v);
39             for (auto e: rg[p.v]) Q.push(node(p.d+e->d, e->u, e));
40         }
41     }
42     heap* merge(heap* curNd, heap* newNd) {
43         if (curNd==nullNd) return newNd;
44         heap* root=new heap; memcpy(root, curNd, sizeof(heap));
45         if (newNd->edge->d < curNd->edge->d) {
46             root->edge=newNd->edge;
47             root->chd[2]=newNd->chd[2];
48             root->chd[3]=newNd->chd[3];
49             newNd->edge=curNd->edge;
50             newNd->chd[2]=curNd->chd[2];
51             newNd->chd[3]=curNd->chd[3];
52         }
53         if (root->chd[0]->dep < root->chd[1]->dep)
54             root->chd[0]=merge(root->chd[0], newNd);
55         else root->chd[1]=merge(root->chd[1], newNd);
56         root->dep=max(root->chd[0]->dep,
57             root->chd[1]->dep)+1;
58         return root;
59     }
60     vector<heap*> V;
61     void build() {
62         nullNd=new heap; nullNd->dep=0; nullNd->edge=new nd;
63         fill(nullNd->chd, nullNd->chd+4, nullNd);
64         while (not dfsQ.empty()) {
65             int u=dfsQ.front(); dfsQ.pop();
66             if (!nxt[u]) head[u]=nullNd;
67             else head[u]=head[nxt[u]->v];
68             V.clear();
69             for (auto&& e: g[u]) {
70                 int v=e->v;
71                 if (dst[v] == -1) continue;
72                 e->d+=dst[v]-dst[u];
73                 if (nxt[u] != e) {
74                     heap* p=new heap; fill(p->chd, p->chd+4, nullNd);
75                     p->dep=1; p->edge=e; V.push_back(p);
76                 }
77             }
78             if (V.empty()) continue;
79             make_heap(V.begin(), V.end(), cmp);
80             #define L(X) ((X<<1)+1)
81             #define R(X) ((X<<1)+2)
82             for (size_t i=0; i<V.size(); i++) {
83                 if (L(i) < V.size()) V[i]->chd[2]=V[L(i)];
84                 else V[i]->chd[2]=nullNd;
85                 if (R(i) < V.size()) V[i]->chd[3]=V[R(i)];
86                 else V[i]->chd[3]=nullNd;
87             }
88             head[u]=merge(head[u], V.front());
89         }
90     }
91     vector<ll> ans;
92     void first_K() {
93         ans.clear(); priority_queue<node> Q;
94         if (dst[s] == -1) return;
95         ans.push_back(dst[s]);
96         if (head[s] != nullNd)
97             Q.push(node(head[s], dst[s]+head[s]->edge->d));
98         for (int _=1; _<k and not Q.empty(); _++) {
99             node p=Q.top(); Q.pop(); ans.push_back(p.d);
100             if (head[p.H->edge->v] != nullNd) {
101                 q.H=head[p.H->edge->v]; q.d=p.d+q.H->edge->d;
102                 Q.push(q);
103             }
104             for (int i=0; i<4; i++)
105                 if (p.H->chd[i] != nullNd) {

```

```

106     q.H=p.H->chd[i];
107     q.d=p.d+p.H->edge->d+p.H->chd[i]->edge->d;
108     Q.push(q);
109 } } }
110 void solve(){ // ans[i] stores the i-th shortest path
111     dijkstra(); build();
112     first_K(); // ans.size() might less than k
113 }
114 } solver;

```

6.9 SCC - Tarjan

```

1 // Author: Ian
2 // tarjan SCC
3 void solve() {
4     V<bool> ins(n, false);
5     V<int> scc(n), dfn(n, -1), low(n, inf);
6     stack<int> s;
7     function<void(int)> dfs = [&](int x) {
8         if (~dfn[x]) return;
9         static int t = 0;
10        dfn[x] = low[x] = t++;
11        s.push(x); ins[x] = true;
12        for (auto i : e[x])
13            if (dfs(i), ins[i])
14                low[x] = min(low[x], low[i]);
15        if (dfn[x] == low[x]) {
16            static int ncnt = 0;
17            int p; do {
18                ins[p = s.top()] = false;
19                s.pop(); scc[p] = ncnt;
20            } while (p != x); ncnt++;
21        }
22    };
23    for (int i = 0; i < n; i++)
24        dfs(i);
25 }

```

6.10 2SAT

```

1 // Author: CryptoGRapheR
2 struct TwoSAT {
3     int n;
4     scc scc;
5     void init(int n) {
6         // (0,1), (2,3), ...
7         n = _n; scc.init(n*2);
8     }
9     void add_disjunction(int a, int na, int b, int nb) {
10        a = 2*a^na, b = 2*b^nb;
11        scc.addEdge(a^1, b);
12        scc.addEdge(b^1, a);
13    }
14    vector<int> solve() {
15        scc.solve();
16        vector<int> assignment(n, 0);
17        for(int i=0; i<n; i++) {
18            if (scc.bl[n*2*i] == scc.bl[n*2*i+1]) return {};
19            assignment[i] = scc.bl[n*2*i] > scc.bl[n*2*i+1];
20        }
21        return assignment;
22    }
23 };

```

7 Tree

7.1 Tree Isomorphism (Rooted Trees)

```

1 // Author: Gino
2 // Checks if 2 rooted trees are isomorphic
3 // Time: O(V log V)
4 // vector<vector<int>> G1, G2;
5 // int root_1, root_2;
6 map<vector<int>, int> dict;
7 int encode(const auto& G, int u, int pa = -1) {
8     vector<int> codes;
9     for (auto& v : G[u])
10        if (v != pa)
11            codes.emplace_back(encode(G, v, u));
12    ranges::sort(codes);
13    auto [it, _] = dict.try_emplace(codes, dict.size());
14    return it->second;
15 }
16 bool isomorphic() {
17     dict.clear(); dict[{}] = 0;
18     return encode(G1, root_1) == encode(G2, root_2);
19 }

```

7.2 Tree Isomorphism (Unrooted Trees)

Find the centroid(s) of T_1, T_2 .

Case 1: T_1, T_2 have different number of centroids $\rightarrow$ NO

Case 2: T_1 has centroid c_1 , T_2 has centroid c_2

$\rightarrow$ rooted_isomorphic(c_1, c_2)

Case 3: T_1 has centroids c_1, c'_1 , T_2 has centroids c_2, c'_2

$\rightarrow$ rooted_isomorphic(c_1, c_2) || rooted_isomorphic(c'_1, c'_2)

7.3 Heavy Light Decomposition

```

1 // Author: Ian
2 void build(V<int>&v);
3 void modify(int p, int k);
4 int query(int ql, int qr);
5 // Insert [ql, qr) segment tree here
6 void solve(){
7     // Input with adjacency list ...
8     V<int> d(n, 0), f(n, 0), sz(n, 1), son(n, -1);
9     F<void(int, int)> dfs1 = [&](int x, int pre) {
10        for (auto i : e[x]) if (i != pre) {
11            d[i] = d[x]+1, f[i] = x;
12            dfs1(i, x), sz[x] += sz[i];
13            if (son[x] == -1 || sz[son[x]] < sz[i])
14                son[x] = i;
15        }
16    }; dfs1(0, 0);
17     V<int> top(n, 0), dfn(n, -1);
18     F<void(int, int)> dfs2 = [&](int x, int t) {
19        static int cnt = 0;
20        dfn[x] = cnt++, top[x] = t;
21        if (son[x] == -1) return;
22        dfs2(son[x], t);
23        for (auto i : e[x]) if (!dfn[i])
24            dfs2(i, i);
25    }; dfs2(0, 0);
26     V<int> dfnv(n);
27     for (int i = 0; i < n; i++)
28         dfnv[dfn[i]] = v[i];
29     build(dfnv);
30     while(q--){
31         int op, a, b, ans; cin >> op >> a >> b;
32         switch(op){
33             case 1:
34                 modify(dfn[a-1], b);
35                 break;
36             case 2:
37                 a--, b--, ans = 0;
38                 while (top[a] != top[b]) {
39                     if (d[top[a]] > d[top[b]]) swap(a, b);
40                     ans = max(ans, query(dfn[top[b]], dfn[b]+1));
41                     b = f[top[b]];
42                 }
43                 if (dfn[a] > dfn[b]) swap(a, b);
44                 ans = max(ans, query(dfn[a], dfn[b]+1));
45                 cout << ans << endl;
46                 break;
47             }
48         }
49 }

```

8 Matching

8.1 Bipartite Matching

```

1 // Author: Gino
2 // Function: Max Bipartite Matching in O(V sqrt(E))
3 // Usage:
4 // >>> init(nx, ny) -> add(x, y (+nx))
5 // >>> hk.max_matching() := the matching plan stores in mx,
6 // my
7 // >>> hk.min_vertex_cover() := the vertex cover plan stores
8 // in vcover
9 // (!) vertices are 0-based: X = [0, nx), Y = [nx, nx+ny)
10 #define pb emplace_back
11 struct HopcroftKarp {
12     int n, nx, ny;
13     vector<vector<int>> G;
14     vector<int> mx, my;
15     void init(int _nx, int _ny) {
16         nx = _nx, ny = _ny;
17         n = nx + ny;
18         G.assign(n, vector<int>());
19     }
20     void add(int x, int y) { G[x].pb(y); G[y].pb(x); }
21     int max_matching() {
22         vector<int> dis, vis;
23         mx.assign(n, -1); my.assign(n, -1);
24         function<bool(int)> dfs = [&](int x) {
25             vis[x] = 1;
26             for (int y : G[x]) {
27                 int p = my[y];
28                 if (p == -1 || (dis[p] == dis[x] + 1 && !vis[p] &&
29                     dfs(p)))
30                     return mx[x] = y, my[y] = x, true;
31             }
32             return false;
33         };
34         for (int x = 0; x < nx; x++)
35             if (mx[x] == -1) dis[x] = 0;
36         while (dfs(0))
37             continue;
38         return count(mx.begin(), mx.end(), -1);
39     }
40 };

```

```

29     return false;
30 };
31 while (true) {
32     dis.assign(n, -1);
33     queue<int> q;
34     for (int x = 0; x < nx; x++)
35         if (mx[x] == -1) dis[x] = 0, q.push(x);
36     while (!q.empty()) {
37         int x = q.front(); q.pop();
38         for (int y : G[x])
39             if (my[y] != -1 && dis[my[y]] == -1)
40                 dis[my[y]] = dis[x] + 1, q.push(my[y]);
41     }
42     vis.assign(n, 0);
43     bool found = false;
44     for (int x = 0; x < nx; x++)
45         if (mx[x] == -1 && dfs(x)) found = true;
46     if (!found) break;
47 }
48 int ans = 0;
49 for (int x = 0; x < nx; x++) if (mx[x] != -1) ans++;
50 return ans;
51 }
52 vector<int> vcover;
53 int min_vertex_cover() {
54     int ans = max_matching();
55     vector<int> vis(n, 0);
56     function<void(int)> dfs = [&](int x) {
57         vis[x] = true;
58         for (int y : G[x]) {
59             if (y == mx[x] || my[y] == -1 || vis[y]) continue;
60             vis[y] = true;
61             dfs(my[y]);
62         }
63     };
64     for (int x = 0; x < nx; x++) if (mx[x] == -1) dfs(x);
65     vcover.clear();
66     for (int x = 0; x < nx; x++) if (!vis[x]) vcover.pb(x);
67     for (int y = nx; y < nx + ny; y++) if (vis[y])
68         vcover.pb(y);
69     return ans;
70 } hk;

```

8.2 Bipartite Weighted Matching

```

1// Author: ckiseki
2// n = #(left vertices) = #(right vertices)
3// left, right vertices labeled 1 ~ n separately
4// 1. build 1-based matrix G of size (n + 1) * (n + 1)
5// G[u][v] := weight between (left u, right v)
6// <!-- if not need perfect matching => set weights of all
   negative edges to 0
7// 2. constrcut (solve at same time): KM km(n, G);
8// => km.ans (max perfect weight sum)
9// => km.fl[u] (left u -> right v)
10// => km.fr[v] (right v -> left u)
11// Time: O(V^3), Space: O(V^2)
12// Test: V <= 500, 243ms
13struct KM { // maximize, test @ UOJ 80
14    int n, l, r; ll ans; // fl and fr are the match
15    vector<ll> hl, hr; vector<int> fl, fr, pre, q;
16    void bfs(const auto &w, int s) {
17        vector<int> vl(n + 1), vr(n + 1); vector<ll> slk(n + 1,
18        LINF);
19        l = r = 0; vr[q[r++] = s] = true;
20        auto check = [&](int x) -> bool {
21            if (vl[x] || slk[x] > 0) return true;
22            vl[x] = true; slk[x] = LINF;
23            if (fl[x] != -1) return (vr[q[r++] = fl[x]] = true);
24            while (x != -1) swap(x, fr[fl[x] = pre[x]]);
25            return false;
26        };
27        while (true) {
28            while (l < r)
29                for (int x = l, y = q[l++]; x <= n; ++x) if (!vl[x])
30                    if (ll val = hl[x] + hr[y] - w[x][y]; val <
31                    slk[x]) {
32                        slk[x] = val;
33                        if (pre[x] = y, !check(x)) return;
34                    }
35            ll d = LINF;
36            for (int x = l; x <= n; ++x) d = min(d, slk[x]);
37            for (int x = l; x <= n; ++x)
38                vl[x] ? hl[x] += d : slk[x] -= d;
39            for (int x = l; x <= n; ++x) if (vr[x]) hr[x] -= d;
40            for (int x = l; x <= n; ++x) if (!check(x)) return;
41        }
42    }
43    KM(int n_, const auto &w) : n(n_), ans(0),

```

```

42     hl(n + 1), hr(n + 1), fl(n + 1, -1), fr(fl), pre(n + 1),
43     q(n + 1) {
44         for (int i = 1; i <= n; ++i) {
45             hl[i] = -LINF;
46             for (int j = 1; j <= n; ++j) hl[i] = max(hl[i],
47             (ll)w[i][j]);
48         }
49         for (int i = 1; i <= n; ++i) bfs(w, i);
50         for (int i = 1; i <= n; ++i) ans += w[i][fl[i]];
51     };

```

8.3 General Matching

```

1// Author: ckiseki
2// 0. build 1-based graph vector<vector<int>> G(n + 1);
3// 1. construct (solve at same time):
4//     GeneralMatching M(G, n); => M.ans
5// status: M.match[x] (== -1 if not matched, 1 <= x <= n)
6// Time: O(V^3), Space: O(V + E)
7// Test: V <= 500, 11ms
8struct GeneralMatching {
9    queue<int> q; int ans, n;
10    vector<int> fa, s, v, pre, match;
11    int Find(int u) {
12        return u == fa[u] ? u : fa[u] = Find(fa[u]);
13    }
14    int LCA(int x, int y) {
15        static int tk = 0; tk++; x = Find(x); y = Find(y);
16        for (; swap(x, y) if (x != 0) {
17            if (v[x] == tk) return x;
18            v[x] = tk;
19            x = Find(pre[match[x]]);
20        }
21    }
22    void Blossom(int x, int y, int l) {
23        for (; Find(x) != l; x = pre[y]) {
24            pre[x] = y, y = match[x];
25            if (s[y] == 1) q.push(y), s[y] = 0;
26            for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
27        }
28    }
29    bool Bfs(auto &&g, int r) {
30        iota(all(fa), 0); ranges::fill(s, -1);
31        q = queue<int>(); q.push(r); s[r] = 0;
32        for (; !q.empty(); q.pop()) {
33            for (int x = q.front(); int u : g[x])
34                if (s[u] == -1) {
35                    if (pre[u] = x, s[u] = 1, match[u] == 0) {
36                        for (int a = u, b = x, last;
37                             b != 0; a = last, b = pre[a])
38                            last = match[b], match[b] = a, match[a] = b;
39                        return true;
40                    }
41                    q.push(match[u]); s[match[u]] = 0;
42                } else if (!s[u] && Find(u) != Find(x)) {
43                    int l = LCA(u, x);
44                    Blossom(x, u, l); Blossom(u, x, l);
45                }
46            }
47        }
48        return false;
49    }
50    GeneralMatching(auto &&g, int n_) : ans(0), n(n_),
51    fa(n + 1), s(n + 1), v(n + 1), pre(n + 1, 0), match(n + 1,
52    0) {
53        for (int x = 1; x <= n; ++x)
54            if (match[x] == 0) ans += Bfs(g, x);
55        for (int x = 1; x <= n; ++x)
56            if (match[x] == 0) match[x] = -1;
57    }
58    }; // tested @ yosupo judge

```

8.4 General Weighted Matching

```

1// Author: ckiseki
2// 1. GeneralWeightedMatching M(n);
3//     (1-based, 1 <= u, v <= n)
4// 2. add edges: M.set_edge(u, v, w);
5// 3. auto [tot_weight, n_matches] = M.solve();
6// 4. M.match[u] (== -1 if not matched, 1-based index if
   matched)
7// Time: O(V^3), Space: O(V^2)
8// Test: V <= 500, 369ms
9struct GeneralWeightedMatching { // 1-based
10    static const int inf = INT_MAX;
11    struct edge { int u, v, w; }; int n, nx;
12    vector<int> lab; vector<vector<edge>> g;
13    vector<int> slack, match, st, pa, S, vis;
14    vector<vector<int>> flo, flo_from; queue<int> q;
15    GeneralWeightedMatching(int n_) : n(n_), nx(n * 2), lab(nx
16    + 1),
17    g(nx + 1, vector<edge>(nx + 1)), slack(nx + 1),

```



```

17     flo[nx + 1], flo_from[nx + 1, vector(n + 1, 0)] {
18         match = st = pa = S = vis = slack;
19         REP(u, 1, n) REP(v, 1, n) g[u][v] = {u, v, 0};
20     }
21     int ED(edge e) {
22         return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
23     void update_slack(int u, int x, int &s) {
24         if (!s || ED(g[u][x]) < ED(g[s][x])) s = u; }
25     void set_slack(int x) {
26         slack[x] = 0;
27         REP(u, 1, n)
28             if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
29                 update_slack(u, x, slack[x]);
30     }
31     void q_push(int x) {
32         if (x <= n) q.push(x);
33         else for (int y : flo[x]) q_push(y);
34     }
35     void set_st(int x, int b) {
36         st[x] = b;
37         if (x > n) for (int y : flo[x]) set_st(y, b);
38     }
39     vector<int> split_flo(auto &f, int xr) {
40         auto it = find(all(f), xr);
41         if (auto pr = it - f.begin(); pr % 2 == 1)
42             reverse(1 + all(f), it = f.end() - pr;
43         auto res = vector(f.begin(), it);
44         return f.erase(f.begin(), it), res;
45     }
46     void set_match(int u, int v) {
47         match[u] = g[u][v].v;
48         if (u <= n) return;
49         int xr = flo_from[u][g[u][v].u];
50         auto &f = flo[u], z = split_flo(f, xr);
51         REP(i, 0, int(z.size())-1) set_match(z[i], z[i ^ 1]);
52         set_match(xr, v); f.insert(f.end(), all(z));
53     }
54     void augment(int u, int v) {
55         for (;;) {
56             int xnv = st[match[u]]; set_match(u, v);
57             if (!xnv) return;
58             set_match(v = xnv, u = st[pa[xnv]]);
59         }
60     }
61     /* SPLIT_HASH_HERE */
62     int lca(int u, int v) {
63         static int t = 0; ++t;
64         for (++t; u || v; swap(u, v)) if (u) {
65             if (vis[u] == t) return u;
66             vis[u] = t; u = st[match[u]];
67             if (u) u = st[pa[u]];
68         }
69         return 0;
70     }
71     void add_blossom(int u, int o, int v) {
72         int b = int(find(n + 1 + all(st), 0) - begin(st));
73         lab[b] = 0, S[b] = 0; match[b] = match[o];
74         vector<int> f = {o};
75         for (int x : {u, v}) {
76             for (int y; x != o; x = st[pa[y]])
77                 f.pb(x), f.pb(y = st[match[x]]), q_push(y);
78             reverse(1 + all(f));
79         }
80         flo[b] = f; set_st(b, b);
81         REP(x, 1, nx) g[b][x].w = g[x][b].w = 0;
82         REP(x, 1, n) flo_from[b][x] = 0;
83         for (int xs : flo[b]) {
84             REP(x, 1, nx)
85                 if (g[b][x].w == 0 || ED(g[xs][x]) < ED(g[b][x]))
86                     g[b][x] = g[xs][x], g[x][b] = g[x][xs];
87             REP(x, 1, n)
88                 if (flo_from[xs][x]) flo_from[b][x] = xs;
89         }
90         set_slack(b);
91     }
92     void expand_blossom(int b) {
93         for (int x : flo[b]) set_st(x, x);
94         int xr = flo_from[b][g[b][pa[b]].u], xs = -1;
95         for (int x : split_flo(flo[b], xr)) {
96             if (xs == -1) { xs = x; continue; }
97             pa[xs] = g[x][xs].u; S[xs] = 1, S[x] = 0;
98             slack[xs] = 0; set_slack(x); q_push(x); xs = -1;
99         }
100         for (int x : flo[b])
101             if (x == xr) S[x] = 1, pa[x] = pa[b];
102             else S[x] = -1, set_slack(x);
103         st[b] = 0;
104     }
105     bool on_found_edge(const edge &e) {

```

```

106         if (int u = st[e.u], v = st[e.v]; S[v] == -1) {
107             int nu = st[match[v]]; pa[v] = e.u; S[v] = 1;
108             slack[v] = slack[nu] = 0; S[nu] = 0; q_push(nu);
109         } else if (S[v] == 0) {
110             if (int o = lca(u, v)) add_blossom(u, o, v);
111             else return augment(u, v), augment(v, u), true;
112         }
113         return false;
114     }
115     /* SPLIT_HASH_HERE */
116     bool matching() {
117         ranges::fill(S, -1); ranges::fill(slack, 0);
118         q = queue<int>();
119         REP(x, 1, nx) if (st[x] == x && !match[x])
120             pa[x] = 0, S[x] = 0, q_push(x);
121         if (q.empty()) return false;
122         for (;;) {
123             while (q.size()) {
124                 int u = q.front(); q.pop();
125                 if (S[st[u]] == 1) continue;
126                 REP(v, 1, n)
127                     if (g[u][v].w > 0 && st[u] != st[v]) {
128                         if (ED(g[u][v]) != 0)
129                             update_slack(u, st[v], slack[st[v]]);
130                         else if (on_found_edge(g[u][v])) return true;
131                     }
132             }
133             int d = inf;
134             REP(b, n + 1, nx) if (st[b] == b && S[b] == 1)
135                 d = min(d, lab[b] / 2);
136             REP(x, 1, nx)
137                 if (int s = slack[x]; st[x] == x && s && S[x] <= 0)
138                     d = min(d, ED(g[s][x]) / (S[x] + 2));
139             REP(u, 1, n)
140                 if (S[st[u]] == 1) lab[u] += d;
141                 else if (S[st[u]] == 0) {
142                     if (lab[u] <= d) return false;
143                     lab[u] -= d;
144                 }
145             REP(b, n + 1, nx) if (st[b] == b && S[b] >= 0)
146                 lab[b] += d * (2 - 4 * S[b]);
147             REP(x, 1, nx)
148                 if (int s = slack[x]; st[x] == x &&
149                     s && st[s] != x && ED(g[s][x]) == 0)
150                     if (on_found_edge(g[s][x])) return true;
151             REP(b, n + 1, nx)
152                 if (st[b] == b && S[b] == 1 && lab[b] == 0)
153                     expand_blossom(b);
154         }
155         return false;
156     }
157     pair<ll, int> solve() {
158         ranges::fill(match, 0);
159         REP(u, 0, n) st[u] = u, flo[u].clear();
160         int w_max = 0;
161         REP(u, 1, n) REP(v, 1, n) {
162             flo_from[u][v] = (u == v ? u : 0);
163             w_max = max(w_max, g[u][v].w);
164         }
165         REP(u, 1, n) lab[u] = w_max;
166         int n_matches = 0; ll tot_weight = 0;
167         while (matching()) ++n_matches;
168         REP(u, 1, n) if (match[u] && match[u] < u)
169             tot_weight += g[u][match[u]].w;
170         REP(u, 1, n) if (match[u] == 0) match[u] = -1;
171         return make_pair(tot_weight, n_matches);
172     }
173     void set_edge(int u, int v, int w) {
174         g[u][v].w = g[v][u].w = w; }
175 }

```

9 Flow

9.1 Flow Methods

- 1 // Author: CRYPTOGRAPHER (some modified by Gino)
- 2 Maximize $c^T x$ subject to $Ax \leq b, x \geq 0$;
- 3 with the corresponding symmetric dual problem,
- 4 Minimize $b^T y$ subject to $A^T y \geq c, y \geq 0$.
- 5
- 6 Maximize $c^T x$ subject to $Ax \leq b$;
- 7 with the corresponding asymmetric dual problem,
- 8 Minimize $b^T y$ subject to $A^T y = c, y \geq 0$.
- 9
- 10 Maximize $\sum x$ subject to $x_i + x_j \leq A_{ij}, x \geq 0$;
- 11 $\Rightarrow$ Maximize $\sum x$ subject to $x_i + x_j \leq A_{ij}$;
- 12 $\Rightarrow$ Minimize $A^T y = \sum A_{ij} y_{ij}$ subject to for all v ,
 $\sum_{i=v \text{ or } j=v} y_{ij} = 1, y_{ij} \geq 0$
- 13 $\Rightarrow$ possible optimal solution: $y_{ij} = \{0, 0.5, 1\}$
- 14 $\Rightarrow y' = 2y$: $\sum_{i=v \text{ or } j=v} y'_{ij} = 2, y'_{ij} = \{0, 1, 2\}$

15=> Minimum Bipartite perfect matching/2 ($V_1=X, V_2=X, E=A$)

16

17 General Graph:

18 |Max Ind. Set| + |Min Vertex Cover| = |V|

19 |Max Ind. Edge Set| + |Min Edge Cover| = |V|

20 Bipartite Graph:

21 |Max Ind. Set| = |Min Edge Cover|

22 |Max Ind. Edge Set| = |Min Vertex Cover|

23

24 To reconstruct the minimum vertex cover, dfs from each

25 unmatched vertex on the left side and with unused edges

26 only. Equivalently, dfs from source with unused edges

27 only and without visiting sink. Then, a vertex is

28 chosen iff. it is on the left side and without visited

29 or on the right side and visited through dfs.

30

31 Minimum Weighted Bipartite Edge Cover:

32 Construct new bipartite graph with $n+m$ vertices on each

33 side:

34 for each vertex u , duplicate a vertex u' on the other side

35 for each edge (u,v,w) , add edges (u,v,w) and (v',u',w)

36 for each vertex u , add edge $(u,u',2w)$ where w is min edge

37 connects to u

38 then the answer is the minimum perfect matching of the new

39 graph (KM)

40

41 Maximum density subgraph ($\sum W_e + \sum W_v$) / |V|

42 Binary search on answer:

43 For a fixed D , construct a Max flow model as follow:

44 Let S be Sum of all weight(or inf)

45 1. from source to each node with cap = S

46 2. For each (u,v,w) in E , $(u \rightarrow v, \text{cap}=w)$, $(v \rightarrow u, \text{cap}=w)$

47 3. For each node v , from v to sink with cap = $S + 2 * D - \text{deg}[v] - 2 * (W \text{ of } v)$

48 where $\text{deg}[v] = \sum \text{weight of edge associated with } v$

49 If $\text{maxflow} < S * |V|$, D is an answer.

50

51 Requiring subgraph: all vertex can be reached from source

52 with

53 edge whose cap > 0 .

54

55 Maximum closed subgraph

56 1. connect source with positive weighted

57 vertex(capacity=weight)

58 2. connect sink with negative weighted vertex(capacity=-

59 weight)

60 3. make capacity of the original edges = inf

61 4. ans = sum(positive weighted vertex weight) - (max flow)

62

63 (Node-disjoint) Min DAG Path Cover (用最少路徑覆蓋所有點)

64 Node disjoint: 拆出來的路徑不能共用同一個點

65 將一個點 u 裂成 $u+$ 和 $u-$, 代表進入和出去

66 對於一條邊 (u, v) 建邊 $u- \Rightarrow v+$

67 1. $+$ 點們和 $-$ 點們形成一張二分圖

68 2. 最差的答案是 n , 代表每個點自己一個點就是一條路徑

69 3. 只要一組 $(u-, v+)$ 匹配成功那對應到的答案恰好會 -1

70 >>> ans = $n -$ 這張二分圖的最大匹配

71

72 General DAG Path Cover

73 跟 Node-disjoint 版本差在點可以被很多條路共用

74 建邊方式 Node-disjoint 差別在條件比較鬆

75 Node-disjoint: $u- \Rightarrow v+$ 只能在 (u, v) 有邊時建

76 General: $u- \Rightarrow v+$ 在 u 能走到 v 的時候就建

77

78 Dilworth Theorem

79 反鏈：一些節點的集合，滿足這些節點互相無法抵達

80

81 最大反鏈 = 最小 General DAG Path Cover

9.2 Dinic

```
1 // Author: Benson (Extensions by Gino)
2 // Function: Max Flow,  $O(V^2 E)$ 
3 // Usage: Call init(n) first, then add(u, v, w) based on
  your model
4 // (!) vertices 0-based
5 // >>> flow() := return max flow
6 // >>> find_cut() := return min cut + store cut set in
  dinic.cut
7 // >>> find_matching() := return |M| + store matching plan
  in dinic.matching
8 // >>> flow_decomposition := return max flow + store
  decomposition in dinic.D
9 #define int long long
10 #define eb emplace_back
11 #define ALL(a) a.begin(), a.end()
12 struct Dinic {
13     struct Edge {
14         // t: to | C: original capacity | c: current capacity |
15         r: residual edge | f: current flow
16         int t, C, c, r, f;
17         bool fw; // is in forward-edge graph
18         Edge() {}
```

```
18         Edge(int _t, int _C, int _r, bool _fw, int _f=0):
19             t(_t), C(_C), c(_C), r(_r), fw(_fw), f(_f) {}
20     };
21     vector<vector<Edge>> G;
22     vector<int> dis, iter;
23     int n, s, t;
24     void init(int _n) {
25         n = _n;
26         G.resize(n), dis.resize(n), iter.resize(n);
27         for(int i = 0; i < n; ++i)
28             G[i].clear();
29     }
30     void add(int a, int b, int c) {
31         G[a].eb(b, c, G[b].size(), true);
32         G[b].eb(a, 0, G[a].size() - 1, false);
33     }
34     bool bfs() {
35         fill(ALL(dis), -1);
36         dis[s] = 0;
37         queue<int> que;
38         que.push(s);
39         while(!que.empty()) {
40             int u = que.front(); que.pop();
41             for(auto& e : G[u]) {
42                 if(e.c > 0 && dis[e.t] == -1) {
43                     dis[e.t] = dis[u] + 1;
44                     que.push(e.t);
45                 }
46             }
47         }
48         return dis[t] != -1;
49     }
50     int dfs(int u, int cur) {
51         if(u == t) return cur;
52         for(int &i = iter[u]; i < (int)G[u].size(); ++i) {
53             auto& e = G[u][i];
54             if(e.c > 0 && dis[u] + 1 == dis[e.t]) {
55                 int ans = dfs(e.t, min(cur, e.c));
56                 if(ans > 0) {
57                     G[e.t][e.r].c += ans;
58                     e.c -= ans;
59                     return ans;
60                 }
61             }
62         }
63         return 0;
64     }
65 // find max flow
66 int flow(int a, int b) {
67     s = a, t = b;
68     int ans = 0;
69     while(bfs()) {
70         fill(ALL(iter), 0);
71         int tmp;
72         while((tmp = dfs(s, INF)) > 0)
73             ans += tmp;
74     }
75     return ans;
76 }
77 // min cut plan
78 vector<pair<pair<int, int>, int>> cut;
79 int find_cut(int a, int b) {
80     int cut_sz = flow(a, b);
81     vector<int> vis(n, 0);
82     cut.clear();
83     function<void(int)> dfs = [&](int u) {
84         vis[u] = 1;
85         for (auto& e : G[u])
86             if (e.c > 0 && !vis[e.t])
87                 dfs(e.t);
88     };
89     dfs(a);
90     for (int u = 0; u < n; u++)
91         if (vis[u])
92             for (auto& e : G[u])
93                 if (!vis[e.t])
94                     cut.eb(make_pair(make_pair(u, e.t), G[e.t]
95 [e.r].c));
96     return cut_sz;
97 }
98 // bipartite matching plan
99 vector<pair<int, int>> matching;
100 int find_matching(int Xstart, int Xend, int Ystart, int
101 Yend, int a, int b) {
102     int msz = flow(a, b);
103     matching.clear();
104     for (int x = Xstart; x <= Xend; x++)
105         for (auto& e : G[x])
106             if (e.c == 0 && Ystart <= e.t && e.t <= Yend)
107                 matching.emplace_back(make_pair(x, e.t));
108     return msz;
109 }
```

```

104 // flow decomposition
105 vector<pair<int, vector<int>>> D; // (flow amount, [path
    pl ... pk])
106 int flow_decomposition(int a, int b) {
107     int mxflow = flow(a, b);
108
109     vector<vector<Edge>> fG(n); // graph consists of forward
    edges
110     for (int u = 0; u < n; u++) {
111         for (auto& e : G[u]) {
112             if (e.fw) {
113                 e.f = e.C - e.c;
114                 if (e.f > 0) fG[u].eb(e);
115             } } }
116
117     vector<int> vis;
118     function<int(int, int)> dfs = [&](int u, int cur) {
119         if (u == b) {
120             D.back().second.eb(u);
121             return cur;
122         }
123         vis[u] = 1;
124         for (auto& e : fG[u]) {
125             if (e.f > 0 && !vis[e.t]) {
126                 int ans = dfs(e.t, min(cur, e.f));
127                 if (ans > 0) {
128                     e.f -= ans;
129                     D.back().second.eb(u);
130                     return ans;
131                 } } }
132         return 0LL;
133     };
134     D.clear();
135     int quota = mxflow;
136     while (quota > 0) {
137         D.emplace_back(make_pair(0, vector<int>()));
138         vis.assign(n, 0);
139         int f = dfs(a, INF);
140         if (f == 0) break;
141         reverse(D.back().second.begin(),
142             D.back().second.end());
143         D.back().first = f, quota -= f;
144     }
145     return mxflow;
146 };

```

9.3 ISAP

```

1 // Author: CRYPTOGRAPHER
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. flow.init(n, s, t);
4 // n = #vertices, s, t = source, sink
5 // 2. flow.addEdge(u, v, c); // u, v in [1, n]
6 // 3. call: int ans = flow.flow();
7 // => ans = max flow value from s to t
8 // Time: O(V^2 * E) worst case, usually much faster in
    practice
9 static const int MAXV=50010;
10 static const int INF =1000000;
11 struct Maxflow{
12     struct Edge{
13         int v,c,r;
14         Edge(int _v,int _c,int _r):v(_v),c(_c),r(_r){}
15     };
16     int s,t; vector<Edge> G[MAXV];
17     int iter[MAXV],d[MAXV],gap[MAXV],tot;
18     void init(int n,int _s,int _t){
19         tot=n,s=_s,t=_t;
20         for(int i=1;i<=tot;i++){
21             G[i].clear(); iter[i]=d[i]=gap[i]=0;
22         }
23     }
24     void addEdge(int u,int v,int c){
25         G[u].push_back(Edge(v,c,SZ(G[v])));
26         G[v].push_back(Edge(u,0,SZ(G[u])-1));
27     }
28     int DFS(int p,int flow){
29         if(p==t) return flow;
30         for(int &i=iter[p];i<SZ(G[p]);i++){
31             Edge &e=G[p][i];
32             if(e.c>0&&d[p]==d[e.v]+1){
33                 int f=DFS(e.v,min(flow,e.c));
34                 if(f){ e.c-=f; G[e.v][e.r].c+=f; return f; }
35             }
36         }
37         if(--gap[d[p]]==0) d[s]=tot;
38         else{ d[p]++; iter[p]=0; ++gap[d[p]]; }
39         return 0;
40     }
41     int flow(){

```

```

42     int res=0;
43     for(res=0,gap[0]=tot;d[s]<tot;res+=DFS(s,INF));
44     return res;
45 }
46 void reset(){
47     for(int i=1;i<=tot;i++) iter[i]=d[i]=gap[i]=0;
48 }
49 } flow;

```

9.4 Bounded Max Flow

```

1 // Author: CRYPTOGRAPHER
2 // Max flow with lower/upper bound on edges (source-sink
    version)
3 // <!-- 1-based: original vertices are numbered 1 ~ n.
4 // Super source, sink: n+1, n+2 respectively.
5 // <!-- dependency: ISAP.cpp
6 // 1. n = #vertices (1 ~ n), m = #edges
7 // s, t = original source, sink (both in [1, n])
8 // 2. l[i], r[i] := edge i goes from l[i] -> r[i] (l[i],
    r[i] in [1, n])
9 // 3. a[i], b[i] := flow on edge i must lie in [a[i], b[i]]
10 // 4. int ans = solve(n, m, s, t);
11 // => ans == -1 : no feasible flow exists
12 // => ans : feasible max flow value
13 // <!-- N, M must be large enough for original graph + 2
    super nodes
14 // Time: O(V^2 E), Space: O(V + E)
15 int in[N], out[N], l[M], r[M], a[M], b[M];
16 int solve(int n, int m, int s, int t){
17     flow.init(n+2, n+1, n+2); // super source = n+1, super
    sink = n+2
18     for(int i = 0; i < m; i++){
19         in[r[i]] += a[i]; out[l[i]] += a[i];
20         flow.addEdge(l[i], r[i], b[i] - a[i]);
21         // flow from l[i] to r[i] must lie in [a[i], b[i]]
22     }
23     int nd = 0;
24     for(int i = 1; i <= n; i++){
25         if(in[i] < out[i]){
26             flow.addEdge(i, flow.t, out[i] - in[i]);
27             nd += out[i] - in[i];
28         }
29         if(out[i] < in[i])
30             flow.addEdge(flow.s, i, in[i] - out[i]);
31     }
32     flow.addEdge(t, s, INF); // original sink -> source
33     if(flow.flow() != nd) return -1; // infeasible
34     int ans = flow.G[s].back().c; // t->s edge's current flow
    = feasible flow s->t
35     flow.G[s].back().c = flow.G[t].back().c = 0;
36     for(size_t i = 0; i < flow.G[flow.s].size(); i++){
37         Maxflow::Edge &e = flow.G[flow.s][i];
38         flow.G[flow.s][i].c = 0; flow.G[e.v][e.r].c = 0;
39     }
40     for(size_t i = 0; i < flow.G[flow.t].size(); i++){
41         Maxflow::Edge &e = flow.G[flow.t][i];
42         flow.G[flow.t][i].c = 0; flow.G[e.v][e.r].c = 0;
43     }
44     flow.addEdge(flow.s, s, INF); flow.addEdge(t, flow.t,
    INF);
45     flow.reset(); // keeps G[] intact, only resets search
    state
46     return ans + flow.flow();
47 }

```

9.5 MCMF

```

1 // Author: CRYPTOGRAPHER
2 // MCMF (supports negative edges without negative cycles)
3 // <!-- 1-based: vertices are numbered 1 ~ n.
4 // 1. MCMF.init(n); // n = #vertices (1 ~ n)
5 // 2. MCMF.add(u, v, cap, cost);
6 // 3. auto [max_flow, min_cost] = MCMF.flow(s, t); // s, t
    in [1, n]
7 // Time: O(F * E), Space: O(V + E)
8 typedef int Tcost;
9 const int MAXV = 20010;
10 const int INFf = 1000000;
11 const Tcost INFc = 1e9;
12 struct MCMF {
13     struct Edge{
14         int v, cap;
15         Tcost w;
16         int rev;
17         bool fw;
18         Edge(){}
19         Edge(int t2, int t3, Tcost t4, int t5, bool t6)
20             : v(t2), cap(t3), w(t4), rev(t5), fw(t6) {}
21     };
22     int V, s, t;
23     vector<Edge> G[MAXV];

```

```

24 void init(int n){
25     V = n;
26     for(int i = 0; i <= V; i++) G[i].clear();
27 }
28 void add(int a, int b, int cap, Tcost w){
29     G[a].push_back(Edge(b, cap, w, (int)G[b].size(), true));
30     G[b].push_back(Edge(a, 0, -w, (int)G[a].size()-1,
31 false));
32 }
33 Tcost d[MAXV];
34 int id[MAXV], inqu[MAXV];
35 bool inqu[MAXV];
36 queue<int> q;
37 pair<int, Tcost> flow(int _s, int _t){
38     s = _s, t = _t;
39     int mxf = 0; Tcost mnc = 0;
40     while(1){
41         fill(d, d+1+V, INFc); // need to use type cast
42         fill(inqu, inqu+1+V, 0);
43         fill(mom, mom+1+V, -1);
44         mom[s] = s;
45         d[s] = 0;
46         q.push(s); inqu[s] = 1;
47         while(q.size()){
48             int u = q.front(); q.pop();
49             inqu[u] = 0;
50             for(int i = 0; i < (int) G[u].size(); i++){
51                 Edge &e = G[u][i];
52                 int v = e.v;
53                 if(e.cap > 0 && d[v] > d[u]+e.w){
54                     d[v] = d[u]+e.w;
55                     mom[v] = u;
56                     id[v] = i;
57                     if(!inqu[v]) q.push(v), inqu[v] = 1;
58                 }
59             }
60             if(mom[t] == -1) break;
61             int df = INFc;
62             for(int u = t; u != s; u = mom[u])
63                 df = min(df, G[mom[u]][id[u]].cap);
64             for(int u = t; u != s; u = mom[u]){
65                 Edge &e = G[mom[u]][id[u]];
66                 e.cap -= df;
67                 G[e.v][e.rev].cap += df;
68             }
69             mxf += df;
70             mnc += df*d[t];
71         }
72         return make_pair(mxf, mnc);
73     }
74 };

```

9.6 Push-Relabel

```

1 // Author: Simon Lindholm (kactl)
2 // 0-based: vertices are numbered 0 ~ n-1
3 // 1. PushRelabel pr(n); // n = #vertices
4 // 2. pr.addEdge(u, v, c); // u, v in [0, n-1]
5 // 3. ll ans = pr.calc(s, t);
6 // => ans = max flow value from s to t
7 // To obtain actual flow:
8 // for (int u = 0; u < n; u++)
9 //     for (auto& e : pr.g[u])
10 //         if (e.f > 0) => (u, e.dest, e.f)
11 // Time:  $O(V^2 * \sqrt{E})$ , better on dense graph
12 struct PushRelabel {
13     struct Edge { int dest, back; ll f, c; };
14     vector<vector<Edge>> g;
15     vector<ll> ec;
16     vector<Edge*> cur;
17     vector<vi> hs; vi H;
18     PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
19
20     void addEdge(int s, int t, ll cap, ll rcap=0) {
21         if (s == t) return;
22         g[s].push_back({t, sz(g[t]), 0, cap});
23         g[t].push_back({s, sz(g[s])-1, 0, rcap});
24     }
25
26     void addFlow(Edge& e, ll f) {
27         Edge &back = g[e.dest][e.back];
28         if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
29         e.f += f; e.c -= f; ec[e.dest] += f;
30         back.f -= f; back.c += f; ec[back.dest] -= f;
31     }
32
33     ll calc(int s, int t) {
34         int v = sz(g); H[s] = v; ec[t] = 1;
35         vi co(2*v); co[0] = v-1;
36         rep(i, 0, v) cur[i] = g[i].data();
37         for (Edge& e : g[s]) addFlow(e, e.c);

```

```

36
37     for (int hi = 0;;) {
38         while (hs[hi].empty()) if (!hi--) return -ec[s];
39         int u = hs[hi].back(); hs[hi].pop_back();
40         while (ec[u] > 0) // discharge u
41             if (cur[u] == g[u].data() + sz(g[u])) {
42                 H[u] = 1e9;
43                 for (Edge& e : g[u]) if (e.c && H[u] >
44 H[e.dest]+1)
45                     H[u] = H[e.dest]+1, cur[u] = &e;
46                 if (++co[H[u]], !--co[hi] && hi < v)
47                     rep(i, 0, v) if (hi < H[i] && H[i] < v)
48                         --co[H[i]], H[i] = v + 1;
49                 hi = H[u];
50             } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
51                 addFlow(*cur[u], min(ec[u], cur[u]->c));
52             else ++cur[u];
53     }
54     bool leftOfMinCut(int a) { return H[a] >= sz(g); }
55 };

```

9.7 Gomory-Hu Tree

```

1 // Author: chilli, Takanori MAEHARA (kactl)
2 // <!> Dependency: PushRelabel.cpp
3 // Given a list of edges representing an undirected flow
4 // graph,
5 // returns edges of the Gomory-Hu tree.
6 // maxflow / mincut of (u, v) => min edge from u to v on
7 // Gomory-Hu tree
8 // Time:  $O(V) * O(\text{PushRelabel})$ 
9 typedef array<ll, 3> Edge; // (u, v, w)
10 vector<Edge> gomoryHu(int N, vector<Edge> ed) {
11     vector<Edge> tree;
12     vi par(N);
13     rep(i, 1, N) {
14         PushRelabel D(N); // Dinic also works
15         for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
16         tree.push_back({i, par[i], D.calc(i, par[i])});
17         rep(j, i+1, N)
18             if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
19     }
20     return tree;
21 }

```

9.8 Global Min Cut

```

1 // Author: ckiseki
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. vector<vector<ll>> G(n+1, vector<ll>(n+1, 0));
4 // 2. add_edge(G, u, v, c); // add undirected edge u-v with
5 // weight c, u,v in [1,n]
6 // 3. ll ans = mincut(G, n); // n = #vertices
7 // => ans = global min cut value
8 // Time:  $O(V^3)$ 
9 void add_edge(auto &w, int u, int v, int c) {
10     w[u][v] += c; w[v][u] += c; }
11 auto phase(const auto &w, int n, vector<int> id) {
12     vector<ll> g(n+1); int s = -1, t = -1;
13     while (!id.empty()) {
14         int c = -1;
15         for (int i : id) if (c == -1 || g[i] > g[c]) c = i;
16         s = t; t = c;
17         id.erase(ranges::find(id, c));
18         for (int i : id) g[i] += w[c][i];
19     }
20     return tuple{s, t, g[t]};
21 }
22 ll mincut(auto w, int n) {
23     ll cut = numeric_limits<ll>::max();
24     vector<int> id(n); iota(all(id), 1);
25     for (int i = 0; i < n - 1; ++i) {
26         auto [s, t, gt] = phase(w, n, id);
27         id.erase(ranges::find(id, t));
28         cut = min(cut, gt);
29         for (int j = 1; j <= n; ++j)
30             w[s][j] += w[t][j], w[j][s] += w[j][t];
31     }
32     return cut;
33 }

```

10 String

10.1 Rolling Hash

```

1 const ll C = 27;
2 inline int id(char c) {return c-'a'+1;}
3 struct RollingHash {
4     string s; int n; ll mod;
5     vector<ll> Cexp, hs;
6     RollingHash(string& _s, ll _mod):
7         s(_s), n((int)_s.size()), mod(_mod)

```



```

8 {
9     Cexp.assign(n, 0);
10    hs.assign(n, 0);
11    Cexp[0] = 1;
12    for (int i = 1; i < n; i++) {
13        Cexp[i] = Cexp[i-1] * C;
14        if (Cexp[i] >= mod) Cexp[i] %= mod;
15    }
16    hs[0] = id(s[0]);
17    for (int i = 1; i < n; i++) {
18        hs[i] = hs[i-1] * C + id(s[i]);
19        if (hs[i] >= mod) hs[i] %= mod;
20    }
21    inline ll query(int l, int r) {
22        ll res = hs[r] - (l ? hs[l-1] * Cexp[r-l+1] : 0);
23        res = (res % mod + mod) % mod;
24        return res; }
25};

```

10.2 KMP

```

1 int n, m;
2 string s, p;
3 vector<int> f;
4 void build() {
5     f.clear(); f.resize(m, 0);
6     int ptr = 0; for (int i = 1; i < m; i++) {
7         while (ptr && p[i] != p[ptr]) ptr = f[ptr-1];
8         if (p[i] == p[ptr]) ptr++;
9         f[i] = ptr;
10    }
11    void init() {
12        cin >> s >> p;
13        n = (int)s.size();
14        m = (int)p.size();
15        build(); }
16    void solve() {
17        int ans = 0, pi = 0;
18        for (int si = 0; si < n; si++) {
19            while (pi && s[si] != p[pi]) pi = f[pi-1];
20            if (s[si] == p[pi]) pi++;
21            if (pi == m) ans++, pi = f[pi-1];
22        }
23    cout << ans << endl; }

```

10.3 Z Value

```

1 string is, it, s;
2 int n; vector<int> z;
3 void init() {
4     cin >> is >> it;
5     s = it+'0'+is;
6     n = (int)s.size();
7     z.resize(n, 0); }
8 void solve() {
9     int ans = 0; z[0] = n;
10    for (int i = 1, l = 0, r = 0; i < n; i++) {
11        if (i <= r) z[i] = min(z[i-l], r-i+1);
12        while (i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++;
13        if (i+z[i]-1 > r) l = i, r = i+z[i]-1;
14        if (z[i] == (int)it.size()) ans++;
15    }
16    cout << ans << endl; }

```

10.4 Manacher

```

1 int n; string S, s;
2 vector<int> m;
3 void manacher() {
4     s.clear(); s.resize(2*n+1, '.');
5     for (int i = 0, j = 1; i < n; i++, j += 2) s[j] = S[i];
6     m.clear(); m.resize(2*n+1, 0);
7     // m[i] := max k such that s[i-k, i+k] is palindrome
8     int mx = 0, mxk = 0;
9     for (int i = 1; i < 2*n+1; i++) {
10        if (mx-(i-mx) >= 0) m[i] = min(m[mx-(i-mx)], mx+mxk-i);
11        while (0 <= i-m[i]-1 && i+m[i]+1 < 2*n+1 &&
12            s[i-m[i]-1] == s[i+m[i]+1]) m[i]++;
13        if (i+m[i] > mx+mxk) mx = i, mxk = m[i];
14    }
15    void init() { cin >> S; n = (int)S.size(); }
16    void solve() {
17        manacher();
18        int mx = 0, ptr = 0;
19        for (int i = 0; i < 2*n+1; i++) if (mx < m[i])
20            { mx = m[i]; ptr = i; }
21        for (int i = ptr-mx; i <= ptr+mx; i++)
22            if (s[i] != '.') cout << s[i];
23    cout << endl; }

```

10.5 Suffix Array

```

1 #define F first

```

```

2 #define S second
3 struct SuffixArray { // don't forget s += "$";
4     int n; string s;
5     vector<int> suf, lcp, rk;
6     vector<int> cnt, pos;
7     vector<pair<pii, int>> buc[2];
8     void init(string _s) {
9         s = _s; n = (int)s.size();
10        // resize(n): suf, rk, cnt, pos, lcp, buc[0~1]
11    }
12    void radix_sort() {
13        for (int t : {0, 1}) {
14            fill(cnt.begin(), cnt.end(), 0);
15            for (auto& i : buc[t]) cnt[(t ? i.F.F : i.F.S) ]++;
16            for (int i = 0; i < n; i++)
17                pos[i] = (!i ? 0 : pos[i-1] + cnt[i-1]);
18            for (auto& i : buc[t])
19                buc[t][i].pos[(t ? i.F.F : i.F.S) ]++ = i;
20        }
21    bool fill_suf() {
22        bool end = true;
23        for (int i = 0; i < n; i++) suf[i] = buc[0][i].S;
24        rk[suf[0]] = 0;
25        for (int i = 1; i < n; i++) {
26            int dif = (buc[0][i].F != buc[0][i-1].F);
27            end &= dif;
28            rk[suf[i]] = rk[suf[i-1]] + dif;
29        } return end;
30    }
31    void sa() {
32        for (int i = 0; i < n; i++)
33            buc[0][i] = make_pair(make_pair(s[i], s[i]), i);
34        sort(buc[0].begin(), buc[0].end());
35        if (fill_suf()) return;
36        for (int k = 0; (1<<k) < n; k++) {
37            for (int i = 0; i < n; i++)
38                buc[0][i] = make_pair(make_pair(rk[i], rk[(i
39                    + (1<<k)) % n]), i);
40            radix_sort();
41            if (fill_suf()) return;
42        }
43        void LCP() { int k = 0;
44            for (int i = 0; i < n-1; i++) {
45                if (rk[i] == 0) continue;
46                int pi = rk[i];
47                int j = suf[pi-1];
48                while (i+k < n && j+k < n && s[i+k] == s[j+k])
49                    k++;
50                lcp[pi] = k;
51                k = max(k-1, 0);
52            }
53    }
54};
55 SuffixArray suffixarray;

```

10.6 Suffix Automaton

```

1 // any path start from root forms a substring of S
2 // occurrence of P: iff SAM can run on input word P
3 // number of different substring: ds[1]-1
4 // total length of all different substring: dsl[1]
5 // max/min length of state i: mx[i]/mx[mom[i]]+1
6 // assume a run on input word P end at state i:
7 // number of occurrences of P: cnt[i]
8 // first occurrence position of P: fp[i]-|P|+1
9 // all position: !clone nodes in dfs from i through rmom
10 const int MXM=1000010;
11 struct SAM{
12     int tot, root, lst, mom[MXM], mx[MXM]; // ind[MXM]
13     int nxt[MXM][33]; // cnt[MXM], ds[MXM], dsl[MXM], fp[MXM]
14     // bool v[MXM], clone[MXN]
15     int newNode(){
16         int res=++tot; fill(nxt[res],nxt[res]+33,0);
17         mom[res]=mx[res]=0; // cnt=ds=ds1=fp=v=clone=0
18         return res;
19     }
20     void init(){ tot=0; root=newNode(); lst=root; }
21     void push(int c){
22         int p=lst, np=newNode(); // cnt[np]=1, clone[np]=0
23         mx[np]=mx[p]+1; // fp[np]=mx[np]-1
24         for (; p && nxt[p][c]==0; p=mom[p]) nxt[p][c]=np;
25         if (p==0) mom[np]=root;
26         else{
27             int q=nxt[p][c];
28             if (mx[p]+1==mx[q]) mom[np]=q;
29             else{
30                 int nq=newNode(); // fp[nq]=fp[q], clone[nq]=1
31                 mx[nq]=mx[p]+1;
32                 for (int i=0; i<33; i++) nxt[nq][i]=nxt[q][i];
33                 mom[nq]=mom[q]; mom[q]=nq; mom[np]=nq;

```

```

34     for(;p&&nxt[p][c]==q;p=mom[p]) nxt[p][c]=nq;
35 }
36 }
37 lst=np;
38 }
39 void calc(){
40     calc(root); iota(ind,ind+tot,1);
41     sort(ind,ind+tot, [&](int i,int j){return mx[i]<mx[j];});
42     for(int i=tot-1;i>=0;i--)
43         cnt[mom[ind[i]]]+=cnt[ind[i]];
44 }
45 void calc(int x){
46     v[x]=ds[x]=1;dsl[x]=0; // rmom[mom[x]].push_back(x);
47     for(int i=0;i<26;i++){
48         if(nxt[x][i]){
49             if(!v[nxt[x][i]]) calc(nxt[x][i]);
50             ds[x]+=ds[nxt[x][i]];
51             dsl[x]+=ds[nxt[x][i]]+dsl[nxt[x][i]];
52         }
53     }
54     void push(char *str){
55         for(int i=0;str[i];i++) push(str[i]-'a');
56     }
57 } sam;

```

10.7 SA-IS

```

1 const int N=300010;
2 struct SA{
3     #define REP(i,n) for(int i=0;i<int(n);i++)
4     #define REP1(i,a,b) for(int i=(a);i<=int(b);i++)
5     bool t[N*2]; int _s[N*2], _sa[N*2];
6     int _c[N*2], x[N], _p[N], _q[N*2], hei[N], r[N];
7     int operator [] (int i){ return _sa[i]; }
8     void build(int *s, int n, int m){
9         memcpy(_s, s, sizeof(int)*n);
10        sais(_s, _sa, _p, _q, _t, _c, n, m); mkhei(n);
11    }
12    void mkhei(int n){
13        REP(i,n) r[_sa[i]]=i;
14        hei[0]=0;
15        REP(i,n) if(r[i]){
16            int ans=i>0?max(hei[r[i-1]]-1,0):0;
17            while(_s[i+ans]==_s[_sa[r[i]-1]+ans]) ans++;
18            hei[r[i]]=ans;
19        }
20    }
21    void sais(int *s, int *sa, int *p, int *q, bool *t, int *c, int
n, int z){
22        bool uniq=t[n-1]=true, neq;
23        int nn=0, nmz=-1, nsa=sa+n, ns=s+n, lst=-1;
24        #define MS0(x,n) memset((x),0,n*sizeof(*(x)))
25        #define MAGIC(XD) MS0(sa,n);\
26        memcpy(x,c,sizeof(int)*z); XD;\
27        memcpy(x+1,c,sizeof(int)*(z-1));\
28        REP(i,n) if(sa[i]&&!t[sa[i]-1]) sa[x[s[sa[i]-1]]]=sa[i]-1;\
29        memcpy(x,c,sizeof(int)*z);\
30        for(int i=n-1;i>=0;i--) if(sa[i]&&t[sa[i]-1]) sa[--
x[s[sa[i]-1]]]=sa[i]-1;
31        MS0(c,z); REP(i,n) uniq&=c[s[i]]<2;
32        REP(i,z-1) c[i+1]=c[i];
33        if(uniq) { REP(i,n) sa[--c[s[i]]]=i; return; }
34        for(int i=n-2;i>=0;i--){
35            t[i]=(s[i]==s[i+1]?t[i+1]:s[i]<s[i+1]);
36            MAGIC(REP1(i,n-1) if(t[i]&&t[i-1]) sa[--
x[s[i]]]=p[q[i]=nn++]=i);
37            REP(i,n) if(sa[i]&&t[sa[i]]&&t[sa[i]-1]){
38                neq=lst<0||memcmp(s+sa[i],s+lst,(p[q[sa[i]]+1]-
sa[i])*sizeof(int));
39                ns[q[lst=sa[i]]]=nmz+=neq;
40            }
41            sais(ns, nsa, p+nn, q+n, t+n, c+z, nn, nmz+1);
42            MAGIC(for(int i=nn-1;i>=0;i--) sa[--
x[s[p[nsa[i]]]]=p[nsa[i]]]);
43        }
44    } sa;
45    int H[N], SA[N], RA[N];
46    void suffix_array(int* ip, int len){
47        // should padding a zero in the back
48        // ip is int array, len is array length
49        // ip[0..n-1] != 0, and ip[len]=0
50        ip[len]=0; sa_build(ip, len, 128);
51        memcpy(H, sa, hei+1, len<2); memcpy(SA, sa, _sa+1, len<2);
52        for(int i=0;i<len;i++) RA[i]=sa.r[i]-1;
53        // resulting height, sa array \in [0,len)
54    }

```

10.8 Minimum Rotation

```

1 // lexicographically minimal rotation
2 // rotate(begin(s), begin(s)+minRotation(s), end(s))
3 int minRotation(string s) {

```

```

4 int a = 0, n = s.size(); s += s;
5 for(int b = 0; b < n; b++) for(int k = 0; k < n; k++) {
6     if(a + k == b || s[a + k] < s[b + k]) {
7         b += max(0, k - 1);
8         break;
9     }
10    if(s[a + k] > s[b + k]) {
11        a = b;
12        break;
13    }
14 }
15 return a;

```

10.9 Aho Corasick

```

1 struct ACautomata{
2     struct Node{
3         int cnt;
4         Node *go[26], *fail, *dic;
5         Node() {
6             cnt = 0; fail = 0; dic = 0;
7             memset(go, 0, sizeof(go));
8         }
9     } pool[1048576], *root;
10    int nMem;
11    Node* new_Node(){
12        pool[nMem] = Node();
13        return &pool[nMem++];
14    }
15    void init() { nMem = 0; root = new_Node(); }
16    void add(const string &str) { insert(root, str, 0); }
17    void insert(Node *cur, const string &str, int pos){
18        for(int i=pos; i<str.size(); i++){
19            if(!cur->go[str[i]-'a'])
20                cur->go[str[i]-'a'] = new_Node();
21            cur=cur->go[str[i]-'a'];
22        }
23        cur->cnt++;
24    }
25    void make_fail(){
26        queue<Node*> que;
27        que.push(root);
28        while (!que.empty()){
29            Node* fr=que.front(); que.pop();
30            for (int i=0; i<26; i++){
31                if (fr->go[i]){
32                    Node *ptr = fr->fail;
33                    while (ptr && !ptr->go[i]) ptr = ptr->fail;
34                    fr->go[i]->fail=ptr=(ptr?ptr->go[i]:root);
35                    fr->go[i]->dic=(ptr->cnt?ptr:ptr->dic);
36                    que.push(fr->go[i]);
37                }
38            }
39        } AC;

```

11 Geometry

11.1 Template

```

1 // Author: Gino
2 typedef long long T;
3 // typedef long double T;
4 const long double eps = 1e-8;
5
6 short sgn(T x) {
7     if (abs(x) < eps) return 0;
8     return x < 0 ? -1 : 1;
9 }
10
11 struct Pt {
12     T x, y;
13     Pt(T _x=0, T _y=0):x(_x), y(_y) {}
14     Pt operator+(Pt a) { return Pt(x+a.x, y+a.y); }
15     Pt operator-(Pt a) { return Pt(x-a.x, y-a.y); }
16     Pt operator*(T a) { return Pt(x*a, y*a); }
17     Pt operator/(T a) { return Pt(x/a, y/a); }
18     T operator*(Pt a) { return x*a.x + y*a.y; }
19     T operator^(Pt a) { return x*a.y - y*a.x; } // 不要反反
20     bool operator<(Pt a) {
21         { return x < a.x || (x == a.x && y < a.y); }
22         //return sgn(x-a.x) < 0 || (sgn(x-a.x) == 0 && sgn(y-a.y) <
0); }
23     bool operator==(Pt a) {
24         { return sgn(x-a.x) == 0 && sgn(y-a.y) == 0; }
25     };
26
27     Pt mv(Pt a, Pt b) { return b-a; }
28     T len2(Pt a) { return a*a; }
29     T dis2(Pt a, Pt b) { return len2(b-a); }
30
31     short ori(Pt a, Pt b) { return ((a^b)>0) - ((a^b)<0); }
32     bool onseg(Pt p, Pt l1, Pt l2) {
33         Pt a = mv(p, l1), b = mv(p, l2);
34         return ((a^b) == 0) && ((a^b) <= 0);

```

35}

11.2 Basic Operations

```
1// Author: Gino
2// Function: Check if a point P sits in a polygon (doesn't
  have to be convex hull)
3// 0 = Bound, 1 = In, -1 = Out
4short inPoly(Pt p) {
5    for (int i = 0; i < n; i++)
6        if (onseg(p, E[i], E[(i+1)%n])) return 0;
7    int cnt = 0;
8    for (int i = 0; i < n; i++)
9        if (banana(p, Pt(p.x+1, p.y+2e9), E[i], E[(i+1)%n]))
10            cnt ^= 1;
11    return (cnt ? 1 : -1);
12}
```

```
1// Author: Gino
2int ud(Pt a) { // up or down half plane
3    if (a.y > 0) return 0;
4    if (a.y < 0) return 1;
5    return (a.x >= 0 ? 0 : 1);
6}
7sort(ALL(E), [&](const Pt& a, const Pt& b){
8    if (ud(a) != ud(b)) return ud(a) < ud(b);
9    return (a^b) > 0;
10});
```

```
1// Author: Gino
2// Function: check if (p1---p2) (q1---q2) banana
3inline bool banana(Pt p1, Pt p2, Pt q1, Pt q2) {
4    if (onseg(p1, q1, q2) || onseg(p2, q1, q2) ||
5        onseg(q1, p1, p2) || onseg(q2, p1, p2)) {
6        return true;
7    }
8    Pt p = mv(p1, p2), q = mv(q1, q2);
9    return (ori(p, mv(p1, q1)) * ori(p, mv(p1, q2)) < 0 &&
10        ori(q, mv(q1, p1)) * ori(q, mv(q1, p2)) < 0);
11}
```

```
1// Author: Gino
2// T: long double
3Pt bananaPoint(Pt p1, Pt p2, Pt q1, Pt q2) {
4    if (onseg(q1, p1, p2)) return q1;
5    if (onseg(q2, p1, p2)) return q2;
6    if (onseg(p1, q1, q2)) return p1;
7    if (onseg(p2, q1, q2)) return p2;
8    double s = abs(mv(p1, p2) ^ mv(p1, q1));
9    double t = abs(mv(p1, p2) ^ mv(p1, q2));
10    return q2 * (s/(s+t)) + q1 * (t/(s+t));
11}

1// Author: Gino
2vector<Pt> hull;
3void convexHull() {
4    hull.clear(); sort(E.begin(), E.end());
5    for (int t : {0, 1}) {
6        int b = (int)hull.size();
7        for (auto& ei : E) {
8            while ((int)hull.size() - b >= 2 &&
9                ori(mv(hull[(int)hull.size()-2],
10                    mv(hull[(int)hull.size()-2], ei)) == -1)
11                {
12                    hull.pop_back();
13                    hull.emplace_back(ei);
14                }
15            hull.pop_back();
16            reverse(E.begin(), E.end());
17        }
18    }
```

```
1// Author: Gino
2// Function: Return doubled area of a polygon
3T dbarea(vector<Pt>& e) {
4    ll res = 0;
5    for (int i = 0; i < (int)e.size(); i++)
6        res += e[i]^e[(i+1)%SZ(e)];
7    return abs(res);
8}
```

11.3 Lower Concave Hull

```
1// Author: Unknown
2struct Line {
3    mutable ll m, b, p;
4    bool operator<(const Line& o) const { return m < o.m; }
5    bool operator<(ll x) const { return p < x; }
6};
7
8struct LineContainer : multiset<Line, less<>> {
9    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
10    const ll inf = LLONG_MAX;
```

```
11    ll div(ll a, ll b) { // floored division
12        return a / b - ((a ^ b) < 0 && a % b); }
13    bool isect(iterator x, iterator y) {
14        if (y == end()) { x->p = inf; return false; }
15        if (x->m == y->m) x->p = x->b > y->b ? inf : -inf;
16        else x->p = div(y->b - x->b, x->m - y->m);
17        return x->p >= y->p;
18    }
19    void add(ll m, ll b) {
20        auto z = insert({m, b, 0}), y = z++, x = y;
21        while (isect(y, z)) z = erase(z);
22        if (x != begin() && isect(--x, y)) isect(x, y =
23            erase(y));
24        while ((y = x) != begin() && (--x->p >= y->p)
25            isect(x, erase(y)));
26    }
27    ll query(ll x) {
28        assert(!empty());
29        auto l = *lower_bound(x);
30        return l.m * x + l.b;
31    }
```

11.4 Pick's Theorem

Consider a polygon which vertices are all lattice points.

Let i = number of points inside the polygon.

Let b = number of points on the boundary of the polygon.

Then we have the following formula:

$$\text{Area} = i + \frac{b}{2} - 1$$

11.5 Minimum Enclosing Circle

```
1// Author: Gino
2// Function: Find Min Enclosing Circle using Randomized O(n)
  Algorithm
3Pt circumcenter(Pt A, Pt B, Pt C) {
4    // a1(x-A.x) + b1(y-A.y) = c1
5    // a2(x-A.x) + b2(y-A.y) = c2
6    // solve using Cramer's rule
7    T a1 = B.x-A.x, b1 = B.y-A.y, c1 = dis2(A, B)/2.0;
8    T a2 = C.x-A.x, b2 = C.y-A.y, c2 = dis2(A, C)/2.0;
9    T D = Pt(a1, b1) ^ Pt(a2, b2);
10    T Dx = Pt(c1, b1) ^ Pt(c2, b2);
11    T Dy = Pt(a1, c1) ^ Pt(a2, c2);
12    if (D == 0) return Pt(-INF, -INF);
13    return A + Pt(Dx/D, Dy/D);
14}
15Pt center; T r2;
16
17void minEncloseCircle() {
18    mt19937
19    gen(chrono::steady_clock::now().time_since_epoch().count());
20    shuffle(ALL(E), gen);
21    center = E[0], r2 = 0;
22
23    for (int i = 0; i < n; i++) {
24        if (dis2(center, E[i]) <= r2) continue;
25        center = E[i], r2 = 0;
26        for (int j = 0; j < i; j++) {
27            if (dis2(center, E[j]) <= r2) continue;
28            center = (E[i] + E[j]) / 2.0;
29            r2 = dis2(center, E[i]);
30            for (int k = 0; k < j; k++) {
31                if (dis2(center, E[k]) <= r2) continue;
32                center = circumcenter(E[i], E[j], E[k]);
33                r2 = dis2(center, E[i]);
34            }
35        }
36    }
```

11.6 PolyUnion

```
1// Author: Unknown
2struct PY{
3    int n; Pt pt[5]; double area;
4    Pt& operator[](const int x){ return pt[x]; }
5    void init(){ //n,pt[0~n-1] must be filled
6        area=pt[n-1]^pt[0];
7        for(int i=0;i<n-1;i++) area+=pt[i]^pt[i+1];
8        if((area/=2)<0)reverse(pt,pt+n),area=-area;
9    }
10};
11PY py[500]; pair<double,int> c[5000];
12inline double segP(Pt &p,Pt &p1,Pt &p2){
13    if(dcmp(p1.x-p2.x)==0) return (p.y-p1.y)/(p2.y-p1.y);
14    return (p.x-p1.x)/(p2.x-p1.x);
```

```

15 }
16 double polyUnion(int n){ //py[0~n-1] must be filled
17 int i,j,ii,jj,ta,tb,r,d; double z,w,s,sum=0,tc,td;
18 for(i=0;i<n;i++) py[i][py[i].n]=py[i][0];
19 for(i=0;i<n;i++){
20 for(ii=0;ii<py[i].n;ii++){
21 r=0;
22 c[r++]=make_pair(0.0,0); c[r++]=make_pair(1.0,0);
23 for(j=0;j<n;j++){
24 if(i==j) continue;
25 for(jj=0;jj<py[j].n;jj++){
26 ta=dcmp(tri(py[i][ii],py[i][ii+1],py[j][jj]));
27 tb=dcmp(tri(py[i][ii],py[i][ii+1],py[j][jj+1]));
28 if(ta==0 && tb==0){
29 if((py[j][jj+1]-py[j][jj])*(py[i][ii+1]-py[i][
30 ii])>0&&<i){
31 c[r++]=make_pair(segP(py[j][jj],py[i][
32 ii],py[i][ii+1]),1);
33 c[r++]=make_pair(segP(py[j][jj+1],py[i][
34 ii],py[i][ii+1]),-1);
35 }else if(ta>=0 && tb<0){
36 tc=tri(py[j][jj],py[j][jj+1],py[i][ii]);
37 td=tri(py[j][jj],py[j][jj+1],py[i][ii+1]);
38 c[r++]=make_pair(tc/(tc+td),1);
39 }else if(ta<0 && tb>=0){
40 tc=tri(py[j][jj],py[j][jj+1],py[i][ii]);
41 td=tri(py[j][jj],py[j][jj+1],py[i][ii+1]);
42 c[r++]=make_pair(tc/(tc+td),-1);
43 } } }
44 sort(c,c+r);
45 z=min(max(c[0].first,0.0),1.0); d=c[0].second; s=0;
46 for(j=1;j<r;j++){
47 w=min(max(c[j].first,0.0),1.0);
48 if(!d) s+=w-z;
49 d+=c[j].second; z=w;
50 }
51 sum+=(py[i][ii]^py[i][ii+1])*s;
52 }
53 return sum/2;

```

11.7 Minkowski Sum

```

1 // Author: Unknown
2 /* convex hull Minkowski Sum*/
3 #define INF 1000000000000000LL
4 int pos( const Pt& tp ){
5 if( tp.Y == 0 ) return tp.X > 0 ? 0 : 1;
6 return tp.Y > 0 ? 0 : 1;
7 }
8 #define N 300030
9 Pt pt[ N ], qt[ N ], rt[ N ];
10 LL Lx,Rx;
11 int dn,un;
12 inline bool cmp( Pt a, Pt b ){
13 int pa=pos( a ),pb=pos( b );
14 if(pa==pb) return (a^b)>0;
15 return pa<pb;
16 }
17 int minkowskiSum(int n,int m){
18 int i,j,r,p,q,fi,fj;
19 for(i=1,p=0;i<n;i++){
20 if( pt[i].Y<pt[p].Y ||
21 (pt[i].Y==pt[p].Y && pt[i].X<pt[p].X) ) p=i; }
22 for(i=1,q=0;i<m;i++){
23 if( qt[i].Y<qt[q].Y ||
24 (qt[i].Y==qt[q].Y && qt[i].X<qt[q].X) ) q=i; }
25 rt[0]=pt[p]+qt[q];
26 r=1; i=p; j=q; fi=fj=0;
27 while(1){
28 if((fj&j==q) ||
29 ( (!fi||i==p) &&
30 cmp(pt[(p+1)%n]-pt[p],qt[(q+1)%m]-qt[q]) ) ){
31 rt[r]=rt[r-1]+pt[(p+1)%n]-pt[p];
32 p=(p+1)%n;
33 fi=1;
34 }else{
35 rt[r]=rt[r-1]+qt[(q+1)%m]-qt[q];
36 q=(q+1)%m;
37 fj=1;
38 }
39 if(r<=1 || ((rt[r]-rt[r-1])^(rt[r-1]-rt[r-2]))!=0) r++;
40 else rt[r-1]=rt[r];
41 if(i==p && j==q) break;
42 }
43 return r-1;
44 }
45 void initInConvex(int n){

```

```

46 int i,p,q;
47 LL Ly,Ry;
48 Lx=INF; Rx=-INF;
49 for(i=0;i<n;i++){
50 if(pt[i].X<Lx) Lx=pt[i].X;
51 if(pt[i].X>Rx) Rx=pt[i].X;
52 }
53 Ly=Ry=INF;
54 for(i=0;i<n;i++){
55 if(pt[i].X==Lx && pt[i].Y<Ly){ Ly=pt[i].Y; p=i; }
56 if(pt[i].X==Rx && pt[i].Y>Ry){ Ry=pt[i].Y; q=i; }
57 }
58 for(dn=0,i=p;i!=q;i=(i+1)%n){ qt[dn++]=pt[i]; }
59 qt[dn]=pt[q]; Ly=Ry=-INF;
60 for(i=0;i<n;i++){
61 if(pt[i].X==Lx && pt[i].Y>Ly){ Ly=pt[i].Y; p=i; }
62 if(pt[i].X==Rx && pt[i].Y>Ry){ Ry=pt[i].Y; q=i; }
63 }
64 for(un=0,i=p;i!=q;i=(i+n-1)%n){ rt[un++]=pt[i]; }
65 rt[un]=pt[q];
66 }
67 inline int inConvex(Pt p){
68 int L,R,M;
69 if(p.X<Lx || p.X>Rx) return 0;
70 L=0;R=dn;
71 while(L<R-1){ M=(L+R)/2;
72 if(p.X<qt[M].X) R=M; else L=M; }
73 if(tri(qt[L],qt[R],p)<0) return 0;
74 L=0;R=un;
75 while(L<R-1){ M=(L+R)/2;
76 if(p.X<rt[M].X) R=M; else L=M; }
77 if(tri(rt[L],rt[R],p)>0) return 0;
78 return 1;
79 }
80 int main(){
81 int n,m,i;
82 Pt p;
83 scanf("%d",&n);
84 for(i=0;i<n;i++) scanf("%lld%lld",&pt[i].X,&pt[i].Y);
85 scanf("%d",&m);
86 for(i=0;i<m;i++) scanf("%lld%lld",&qt[i].X,&qt[i].Y);
87 n=minkowskiSum(n,m);
88 for(i=0;i<n;i++) pt[i]=rt[i];
89 scanf("%d",&m);
90 for(i=0;i<m;i++) scanf("%lld%lld",&qt[i].X,&qt[i].Y);
91 n=minkowskiSum(n,m);
92 for(i=0;i<n;i++) pt[i]=rt[i];
93 initInConvex(n);
94 scanf("%d",&m);
95 for(i=0;i<m;i++){
96 scanf("%lld %lld",&p.X,&p.Y);
97 p.X*=3; p.Y*=3;
98 puts(inConvex(p)?"YES":"NO");
99 }
100 }

```

12 Number Theory

12.1 Prime Sieve and Defactor

```

1 // Author: Gino
2 const int maxc = 1e6 + 1;
3 vector<int> lpf;
4 vector<int> prime;
5
6 void seive() {
7 prime.clear();
8 lpf.resize(maxc, 1);
9 for (int i = 2; i < maxc; i++) {
10 if (lpf[i] == 1) {
11 lpf[i] = i;
12 prime.emplace_back(i);
13 }
14 for (auto& j : prime) {
15 if (i * j >= maxc) break;
16 lpf[i * j] = j;
17 if (j == lpf[i]) break;
18 } } }
19 vector<pii> fac;
20 void defactor(int u) {
21 fac.clear();
22 while (u > 1) {
23 int d = lpf[u];
24 fac.emplace_back(make_pair(d, 0));
25 while (u % d == 0) {
26 u /= d;
27 fac.back().second++;
28 } } }

```


12.2 Harmonic Series

```
1// Author: Gino
2// O(n log n)
3for (int i = 1; i <= n; i++) {
4    for (int j = i; j <= n; j += i) {
5        // O(1) code
6    }
7}
8
9// PIE
10// given array a[0], a[1], ..., a[n - 1]
11// calculate dp[x] = number of pairs (a[i], a[j]) such that
12// gcd(a[i], a[j]) = x // (i < j)
13//
14// idea: let mc(x) = # of y s.t. x|y
15// f(x) = # of pairs s.t. gcd(a[i], a[j]) >= x
16// f(x) = C(mc(x), 2)
17// dp[x] = f(x) - sum(dp[y], x < y and x|y)
18const int maxc = 1e6;
19vector<int> cnt(maxc + 1, 0), dp(maxc + 1, 0);
20for (int i = 0; i < n; i++)
21    cnt[a[i]]++;
22
23for (int x = maxc; x >= 1; x--) {
24    ll cnt_mul = 0; // number of multiples of x
25    for (int y = x; y <= maxc; y += x)
26        cnt_mul += cnt[y];
27
28    dp[x] = cnt_mul * (cnt_mul - 1) / 2; // number of pairs
29    // that are divisible by x
30    for (int y = x + x; y <= maxc; y += x)
31        dp[x] -= dp[y]; // PIE: subtract all dp[y] for y >
32    // x and x|y
33}
```

12.3 Count Number of Divisors

```
1// Author: Gino
2// Function: Count the number of divisors for all x <= 10^6
3// using harmonic series
4const int maxc = 1e6;
5vector<int> facs;
6
7void find_all_divisors() {
8    facs.clear(); facs.resize(maxc + 1, 0);
9    for (int x = 1; x <= maxc; x++) {
10        for (int y = x; y <= maxc; y += x) {
11            facs[y]++;
12        }
13}
```

12.4 數論分塊

```
1// Author: Gino
2/*
3n = 17
4i: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
5n/i: 17 8 5 4 3 2 2 2 1 1 1 1 1 1 1 1 1
6
7          ^       ^
8          L(2)   R(2)
9
10L(x) := left bound for n/i = x
11R(x) := right bound for n/i = x
12
13===== FORMULA =====
14>>> R = n / (n/L) <<<
15=====
16Example: L(2) = 6
17           R(2) = 17 / (17 / 6)
18           = 17 / 2
19           = 8
20*/
21// ===== CODE =====
22
23for (ll l = 1, r = 1, q = n; l <= n; l = r + 1) {
24    q = n/l;
25    r = n/q;
26    // Process your code here
27}
28// q, l, r: 17 1 1
29// q, l, r: 8 2 2
30// q, l, r: 5 3 3
31// q, l, r: 4 4 4
32// q, l, r: 3 5 5
33// q, l, r: 2 6 8
34// q, l, r: 1 9 17
```

12.5 Pollard's rho

```
1// Author: Unknown
```

```
2// Function: Find a non-trivial factor of a big number in
3// O(n^(1/4) log^2(n))
4ll find_factor(ll number) {
5    __int128 x = 2;
6    for (__int128 cycle = 1; ; cycle++) {
7        __int128 y = x;
8        for (int i = 0; i < (1<<cycle); i++) {
9            x = (x * x + 1) % number;
10           __int128 factor = __gcd(x - y, number);
11           if (factor > 1)
12               return factor;
13        }
14    }
15}
```

```
1# Author: Unknown
2# Function: Find a non-trivial factor of a big number in
3// O(n^(1/4) log^2(n))
4from itertools import count
5from math import gcd
6from sys import stdin
7
8for s in stdin:
9    number, x = int(s), 2
10   brk = False
11   for cycle in count(1):
12       y = x
13       if brk:
14           break
15       for i in range(1<< cycle):
16           x = (x * x + 1) % number
17           factor = gcd(x - y, number)
18           if factor > 1:
19               print(factor)
20               brk = True
21               break
```

12.6 Miller Rabin

```
1// Author: Unknown, Modified by Gino
2// Function: Check if a number is a prime in O(100 *
3// log^2(n))
4// miller_rabin(): return 1 if prime, 0 otherwise
5inline ll mul(ll x, ll y, ll mod) {
6    return (__int128)(x) * y % mod;
7}
8ll mypow(ll a, ll b, ll mod) {
9    ll r = 1;
10   while (b > 0) {
11       if (b & 1) r = mul(r, a, mod);
12       a = mul(a, a, mod);
13       b >>= 1;
14   }
15   return r;
16}
17bool witness(ll a, ll n, ll u, int t){
18    ll x = mypow(a, u, n);
19    for(int i = 0; i < t; i++) {
20        ll nx = mul(x, x, n);
21        if (nx == 1 && x != 1 && x != n-1) return true;
22        x = nx;
23    }
24    return x != 1;
25}
26bool miller_rabin(ll n) {
27    // if n >= 3,474,749,660,383
28    // change {2, 3, 5, 7, 11, 13} to
29    // {2, 325, 9375, 28178, 450775, 9780504, 1795265022}
30    if (n < 2) return false;
31    if (!(n & 1)) return n == 2;
32    ll u = n - 1; int t = 0;
33    while (!(u & 1)) u >>= 1, t++;
34    for (ll a : {2, 3, 5, 7, 11, 13}) {
35        if (a % n == 0) continue;
36        if (witness(a, n, u, t)) return false;
37    }
38    return true;
39}
40// bases that make sure no pseudoprimes flee from test
41// if WA, replace randll(n - 1) with these bases:
42// n < 4,759,123,141          3 : 2, 7, 61
43// n < 1,122,004,669,633     4 : 2, 13, 23, 1662803
44// n < 3,474,749,660,383     6 : pirms <= 13
45// n < 2^64                  7 :
46// 2, 325, 9375, 28178, 450775, 9780504, 1795265022
```

12.7 Discrete Log

```
1// Author: ckiseki
2// find x^? = y (mod M)
3// O(sqrt(M))
```

```

4 template<typename Int>
5 Int BSGS(Int x, Int y, Int M) {
6     // x^? ≡ y (mod M)
7     Int t = 1, c = 0, g = 1;
8     for (Int M_ = M; M_ > 0; M_ >= 1) g = g * x % M;
9     for (g = gcd(g, M); t % g != 0; ++c) {
10         if (t == y) return c;
11         t = t * x % M;
12     }
13     if (y % g != 0) return -1;
14     t /= g, y /= g, M /= g;
15     Int h = 0, gs = 1;
16     for (; h * h < M; ++h) gs = gs * x % M;
17     unordered_map<Int, Int> bs;
18     for (Int s = 0; s < h; bs[y] = ++s) y = y * x % M;
19     for (Int s = 0; s < M; s += h) {
20         t = t * gs % M;
21         if (bs.count(t)) return c + s + h - bs[t];
22     }
23     return -1;
24 }

```

12.8 Discrete Sqrt

```

1 // Author: ckiseki
2 // find solution for x^2 = n (mod P)
3 // O(log^2 P)
4 int get_root(int n, int P) { // ensure 0 <= n < P
5     if (P == 2 || n == 0) return n;
6     auto check = [&](ll x) {
7         return modpow(x, (P - 1) / 2, P);
8     };
9     if (check(n) != 1) return -1;
10    mt19937 rnd(7122); ll z = 1, w;
11    while (check(w = (z * z - n + P) % P) != P - 1)
12        z = rnd() % P;
13    const auto M = [P, w](auto &u, auto &v) {
14        auto [a, b] = u; auto [c, d] = v;
15        return make_pair((a * c + b * d % P * w) % P,
16                          (a * d + b * c) % P);
17    };
18    pair<ll, ll> r(1, 0), e(z, 1);
19    for (int q = (P + 1) / 2; q; q >= 1, e = M(e, e))
20        if (q & 1) r = M(r, e);
21    return int(r.first); // sqrt(n) mod P where P is prime

```

12.9 Fast Power

Note: $a^n \equiv a^{(n \bmod (p-1))} \pmod{p}$

12.10 Extend GCD

```

1 // Author: Gino
2 // [Usage]
3 // bezout(a, b, c):
4 //     find solution to ax + by = c
5 //     return {-LINF, -LINF} if no solution
6 // inv(a, p):
7 //     find modulo inverse of a under p
8 //     return -1 if not exist
9 // CRT(vector<ll>& a, vector<ll>& m)
10 //     find a solution pair (x, mod) satisfies all x = a[i]
11 //         (mod m[i])
12 //     return {-LINF, -LINF} if no solution
13 const ll LINF = 4e18;
14 typedef pair<ll, ll> pll;
15 template<typename T1, typename T2>
16 T1 chmod(T1 a, T2 m) {
17     return (a % m + m) % m;
18 }
19
20 ll GCD;
21 pll extgcd(ll a, ll b) {
22     if (b == 0) {
23         GCD = a;
24         return pll{1, 0};
25     }
26     pll ans = extgcd(b, a % b);
27     return pll{ans.second, ans.first - a/b * ans.second};
28 }
29 pll bezout(ll a, ll b, ll c) {
30     bool negx = (a < 0), negy = (b < 0);
31     pll ans = extgcd(abs(a), abs(b));
32     if (c % GCD != 0) return pll{-LINF, -LINF};
33     return pll{ans.first * c/GCD * (negx ? -1 : 1),
34               ans.second * c/GCD * (negy ? -1 : 1)};
35 }
36 ll inv(ll a, ll p) {
37     if (p == 1) return -1;
38     pll ans = bezout(a % p, -p, 1);
39     if (ans == pll{-LINF, -LINF}) return -1;
40     return chmod(ans.first, p);

```

```

41 }
42 pll CRT(vector<ll>& a, vector<ll>& m) {
43     for (int i = 0; i < (int)a.size(); i++)
44         a[i] = chmod(a[i], m[i]);
45
46     ll x = a[0], mod = m[0];
47     for (int i = 1; i < (int)a.size(); i++) {
48         pll sol = bezout(mod, m[i], a[i] - x);
49         if (sol.first == -LINF) return pll{-LINF, -LINF};
50
51         // prevent long long overflow
52         ll p = chmod(sol.first, m[i] / GCD);
53         ll lcm = mod / GCD * m[i];
54         x = chmod((__int128)p * mod + x, lcm);
55         mod = lcm;
56     }
57     return pll{x, mod};
58 }

```

12.11 Mu + Phi

```

1 // Author: Gino
2 const int maxn = 1e6 + 5;
3 ll f[maxn];
4 vector<int> lpf, prime;
5 void build() {
6     lpf.clear(); lpf.resize(maxn, 1);
7     prime.clear();
8     f[1] = ...; /* mu[1] = 1, phi[1] = 1 */
9     for (int i = 2; i < maxn; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i; prime.emplace_back(i);
12             f[i] = ...; /* mu[i] = 1, phi[i] = i-1 */
13         }
14         for (auto& j : prime) {
15             if (i*j >= maxn) break;
16             lpf[i*j] = j;
17             if (i % j == 0) f[i*j] = ...; /* 0, phi[i]*j */
18             else f[i*j] = ...; /* -mu[i], phi[i]*phi[j] */
19             if (j >= lpf[i]) break;
20         }

```

12.12 Other Formulas

- Pisano Period: 任何線性遞迴 (比如費氏數列) 模任何一個數字 M 都會循環, 找循環節 $\pi(M)$ 先質因數分解 $M = \prod p_i^{e_i}$, 然後 $\pi(M) = \text{lcm}(\pi(p_i^{e_i}))$,
- Inversion: $aa^{-1} \equiv 1 \pmod{m}$. a^{-1} exists iff $\gcd(a, m) = 1$.
- Linear inversion: $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \bmod a)^{-1} \pmod{m}$
- Fermat's little theorem: $a^p \equiv a \pmod{p}$ if p is prime.
- Euler function: $\varphi(n) = n \prod_{p|n} \frac{p-1}{p}$
- Euler theorem: $a^{\varphi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$. If a, n are not coprime: 質因數分解 $n = \prod p_i^{e_i}$, 對每個 $p_i^{e_i}$ 分開看他們跟 a 是否互質 (互質: Fermat/不互質: 夠大的指數會直接割成 0), 最後用 CRT 合併.
- Extended Euclidean algorithm: $ax + by = \gcd(a, b) = \gcd(b, a \bmod b) = \gcd(b, a - \lfloor \frac{a}{b} \rfloor b) = bx_1 + (a - \lfloor \frac{a}{b} \rfloor b)y_1 = ay_1 + b(x_1 - \lfloor \frac{a}{b} \rfloor y_1)$
- Divisor function: $\sigma_x(n) = \sum_{d|n} d^x$. $n = \prod_{i=1}^r p_i^{a_i}$. $\sigma_x(n) = \prod_{i=1}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$ if $x \neq 0$. $\sigma_0(n) = \prod_{i=1}^r (a_i + 1)$.
- Chinese remainder theorem (Coprime Moduli): $x \equiv a_i \pmod{m_i}$. $M = \prod m_i$. $M_i = \frac{M}{m_i}$. $t_i = M_i^{-1}$. $x = kM + \sum a_i t_i M_i$, $k \in \mathbb{Z}$.
- Chinese remainder theorem: $x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2} \Rightarrow x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$. Solve for (p, q) using ExtGCD. $x \equiv m_1 p + a_1 \equiv m_2 q + a_2 \pmod{\text{lcm}(m_1, m_2)}$
- Avoiding Overflow: $ca \bmod cb = c(a \bmod b)$
- Dirichlet Convolution: $(f * g)(n) = \sum_{d|n} f(d)g(\frac{n}{d})$
- Important Multiplicative Functions + Properties:
 - $\varepsilon(n) = [n = 1]$
 - $1(n) = 1$
 - $\text{id}(n) = n$
 - $\mu(n) = 0$ if n has squared prime factor

$$5. \mu(n) = (-1)^k \text{ if } n = p_1 p_2 \cdots p_k$$

$$6. \varepsilon = \mu * 1$$

$$7. \varphi = \mu * id$$

$$8. [n=1] = \sum_{d|n} \mu(d)$$

$$9. [gcd=1] = \sum_{d|gcd} \mu(d)$$

$$\bullet \text{ Möbius inversion: } f = g * 1 \Leftrightarrow g = f * \mu$$

12.13 Polynomial

```
1// Author: Gino
2// Preparation: first set_mod(mod, g), then init_ntt()
3// everytime you change modulus, you have to call init_ntt()
  again
4//
5// [Usage]
6// polynomial: vector<ll> a, b
7// negation: -a
8// add/subtract: a += b, a -= b
9// convolution: a *= b
10// in-place modulo: mod(a, b)
11// in-place inversion under mod x^N: inv(ia, N)
12
13
14const int maxk = 20;
15const int maxn = 1<<maxk;
16
17using u64 = unsigned long long;
18using u128 = __uint128_t;
19
20int g;
21u64 MOD;
22u64 BARRETT_IM; // [ 2^64 / MOD ]
23
24inline void set_mod(u64 m, int _g) {
25    g = _g;
26    MOD = m;
27    BARRETT_IM = (u128(1) << 64) / m;
28}
29inline u64 chmod(u128 x) {
30    u64 q = (u64)((x * BARRETT_IM) >> 64);
31    u64 r = (u64)(x - (u128)q * MOD);
32    if (r >= MOD) r -= MOD;
33    return r;
34}
35inline u64 mmul(u64 a, u64 b) {
36    return chmod((u128)a * b);
37}
38ll pw(ll a, ll n) {
39    ll ret = 1;
40    while (n > 0) {
41        if (n & 1) ret = mmul(ret, a);
42        a = mmul(a, a);
43        n >>= 1;
44    }
45    return ret;
46}
47
48vector<ll> X, iX;
49vector<int> rev;
50void init_ntt() {
51    X.assign(maxn, 1); // x1 = g^((p-1)/n)
52    iX.assign(maxn, 1);
53
54    ll u = pw(g, (MOD-1)/maxn);
55    ll iu = pw(u, MOD-2);
56    for (int i = 1; i < maxn; i++) {
57        X[i] = mmul(X[i-1], u);
58        iX[i] = mmul(iX[i-1], iu);
59    }
60
61    if ((int)rev.size() == maxn) return;
62    rev.assign(maxn, 0);
63    for (int i = 1, hb = -1; i < maxn; i++) {
64        if (!(i & (i-1))) hb++;
65        rev[i] = rev[i ^ (1<<hb)] | (1<<(maxk-hb-1));
66    }
67}
68template<typename T>
69void NTT(vector<T>& a, bool inv=false) {
70    int _n = (int)a.size();
71    int k = __lg(_n) + ((1<<__lg(_n)) != _n);
72    int n = 1<<k;
73    a.resize(n, 0);
74
75    short shift = maxk-k;
76    for (int i = 0; i < n; i++)
77        if (i > (rev[i]>>shift))
78            swap(a[i], a[rev[i]>>shift]);
79    for (int len = 2, half = 1, div = maxn>>1; len <= n;
80        len<<=1, half<<=1, div>>=1) {
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164

```

```
79    for (int i = 0; i < n; i += len) {
80        for (int j = 0; j < half; j++) {
81            T u = a[i+j];
82            T v = mmul(a[i+j+half], (inv ? iX[j*div] :
83                X[j*div]));
84            a[i+j] = (u+v >= MOD ? u+v-MOD : u+v);
85            a[i+j+half] = (u-v < 0 ? u-v+MOD : u-v);
86        } }
87    if (inv) {
88        T dn = pw(n, MOD-2);
89        for (auto& x : a) {
90            x = mmul(x, dn);
91        } }
92    template<typename T>
93    inline void shrink(vector<T>& a) {
94        int cnt = (int)a.size();
95        for (; cnt > 0; cnt--) if (a[cnt-1]) break;
96        a.resize(max(cnt, 1));
97    }
98    template<typename T>
99    vector<T>& operator*=(vector<T>& a, vector<T> b) {
100        int na = (int)a.size();
101        int nb = (int)b.size();
102        a.resize(na + nb - 1, 0);
103        b.resize(na + nb - 1, 0);
104
105        NTT(a); NTT(b);
106        for (int i = 0; i < (int)a.size(); i++)
107            a[i] = mmul(a[i], b[i]);
108        NTT(a, true);
109
110        shrink(a);
111        return a;
112    }
113    inline ll crt(ll a0, ll a1, ll m1, ll m2, ll inv_m1_mod_m2){
114        // x ≡ a0 (mod m1), x ≡ a1 (mod m2)
115        // t = (a1 - a0) * inv(m1) mod m2
116        // x = a0 + t * m1 (mod m1*m2)
117        ll t = chmod(a1 - a0);
118        if (t < 0) t += m2;
119        t = (ll)((__int128)t * inv_m1_mod_m2 % m2);
120        return a0 + (ll)((__int128)t * m1);
121    }
122    void mul_crt() {
123        // a copy to a1, a2 | b copy to b1, b2
124        ll M1 = 998244353, M2 = 1004535809;
125        g = 3; set_mod(M1); init_ntt(); a1 *= b1;
126        g = 3; set_mod(M2); init_ntt(); a2 *= b2;
127
128        ll inv_m1_mod_m2 = pw(M1, M2 - 2);
129        for (int i = 2; i <= 2 * k; i++)
130            cout << crt(a1[i], a2[i], M1, M2, inv_m1_mod_m2) <<
131            '\n';
132        cout << endl;
133    }
134}
135
136/* P = r*2^k + 1
137P
138r    k    g
139119 23  3
1401004535809 479 21  3
141
142P
143r    k    g
1441    1    2
1451    2    2
1461    4    3
1473    5    5
1483    6    5
1491    8    3
15015    9    17
1513    12    11
1525    13    3
1531    16    3
1543    18    10
15511    19    3
1567    20    3
15711    21    3
15825    22    3
1595    25    3
1607    26    3
161479 21    3
16215    27    31
16317    27    3
1643    30    5
16535    31    3
1669    33    7
1673    36    22
16815    37    7
1695    39    3

```

```

165 6597069766657      3  41  5
166 39582418599937     9  42  5
167 79164837199873     9  43  5
168 263882790666241    15  44  7
169 1231453023109121   35  45  3
170 1337006139375617   19  46  3
171 3799912185593857   27  47  5
172 4222124650659841   15  48  19
173 7881299347898369    7  50  6
174 31525197391593473    7  52  3
175 180143985094819841  5  55  6
176 1945555039024054273 27  56  5
177 4179340454199820289 29  57  3
178 9097271247288401921 505 54  6 */

```

12.14 Counting Primes

```

1 // prime_count - #primes in [1..n] ( $O(n^{2/3})$  time,
2 //  $O(\sqrt{n})$  memory)
3 using u64 = unsigned long long;
4 static inline u64 prime_count(u64 n){
5     if(n<=1) return 0;
6     int v = (int)floor(sqrt((long double)n));
7     int s = (v+1)>>1, pc = 0;
8     vector<int> smalls(s), roughs(s), skip(v+1);
9     vector<long long> larges(s);
10
11     for(int i=0;i<s;++i){
12         smalls[i]=i;
13         roughs[i]=2*i+1;
14         larges[i]=(long long)((n/roughs[i]-1)>>1);
15     }
16
17     for(int p=3;p<=v;p+=2) if(!skip[p]){
18         int q = p*p;
19         if(1LL*q*q > (long long)n) break;
20         skip[p]=1;
21         for(int i=q;i<=v;i+=2*p) skip[i]=1;
22
23         int ns=0;
24         for(int k=0;k<s;++k){
25             int i = roughs[k];
26             if(skip[i]) continue;
27             u64 d = (u64)i * (u64)p;
28             long long sub = (d <= (u64)v)
29                 ? larges[smalls[(int)(d>>1)] - pc]
30                 : smalls[(int)((n/d - 1) >> 1)];
31             larges[ns] = larges[k] - sub + pc;
32             roughs[ns++] = i;
33         }
34         s = ns;
35         for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
36             int c = smalls[j>>1] - pc;
37             for(int e=(j*p)>>1; i>=e; --i) smalls[i] -= c;
38         }
39         ++pc;
40     }
41
42     larges[0] += 1LL*(s + 2*(pc-1))*(s-1) >> 1;
43     for(int k=1;k<s;++k) larges[0] -= larges[k];
44
45     for(int l=1;l<s;++l){
46         int q = roughs[l];
47         u64 m = n / (u64)q;
48         long long t = 0;
49         int e = smalls[(int)((m/q - 1) >> 1)] - pc;
50         if(e < l+1) break;
51         for(int k=l+1;k<=e;++k) t += smalls[(int)((m/
52 (u64)roughs[k] - 1) >> 1)];
53         larges[0] += t - 1LL*(e - l)*(pc + l - 1);
54     }
55     return (u64)(larges[0] + 1);
56 }

```

12.15 Linear Sieve for Other Number Theoretic Functions

```

1 // linear_sieve(n, primes, lp, phi, mu, d, sigma)
2 // Outputs over the index range 0..n (n >= 1):
3 //   primes : all primes in [2..n], increasing.
4 //   lp      : lowest prime factor; lp[1]=0, lp[x] is the
5 //             smallest prime dividing x.
6 //   phi      : Euler totient, phi[x] =  $\prod_{1 \leq k \leq x : \gcd(k,x)=1} k$ . Multiplicative.
7 //   mu       : Möbius; mu[1]=1, mu[x]=0 if x has a squared
8 //             prime factor, else  $(-1)^{\#\text{distinct primes}}$ .
9 //   d         : number of divisors; if  $x = \prod p_i^{e_i}$ , then
10 //              $d[x] = \prod (e_i + 1)$ . Multiplicative.
11 //   sigma     : sum of divisors; if  $x = \prod p_i^{e_i}$ , then
12 //              $\sigma[x] = \prod (1 + p_i + \dots + p_i^{e_i})$ . (use 1LL)
13 //
14 //

```

```

10 // Complexity:  $O(n)$  time,  $O(n)$  memory.
11 // Notes: Arrays are resized inside; primes is cleared and
12 //         reserved. sigma uses ll to avoid 32-bit overflow.
13 static inline void linear_sieve(
14     int n,
15     std::vector<int> &primes,
16     std::vector<int> &lp,
17     std::vector<int> &phi,
18     std::vector<int> &mu,
19     std::vector<int> &d,
20     std::vector<ll> &sigma
21 ) {
22     lp.assign(n + 1, 0); phi.assign(n + 1, 0); mu.assign(n +
23 1, 0); d.assign(n + 1, 0); sigma.assign(n + 1, 0);
24     primes.clear(); primes.reserve(n > 1 ? n / 10 : 0);
25     std::vector<int> cnt(n + 1, 0), core(n + 1, 1);
26     std::vector<ll> p_pow(n + 1, 1), sum_p(n + 1, 1);
27     phi[1] = mu[1] = d[1] = sigma[1] = 1;
28
29     for (int i = 2; i <= n; ++i) {
30         if (!lp[i]) {
31             lp[i] = i; primes.push_back(i);
32             phi[i] = i - 1; mu[i] = -1; d[i] = 2;
33             cnt[i] = 1; p_pow[i] = i; core[i] = 1;
34             sum_p[i] = 1 + (ll)i; sigma[i] = sum_p[i];
35
36             for (int p : primes) {
37                 long long ip = 1LL * i * p;
38                 if (ip > n) break;
39                 lp[ip] = p;
40                 if (p == lp[i]) {
41                     cnt[ip] = cnt[i] + 1; p_pow[ip] = p_pow[i] * p;
42                     core[ip] = core[i];
43                     sum_p[ip] = sum_p[i] + p_pow[ip];
44                     phi[ip] = phi[i] * p; mu[ip] = 0;
45                     d[ip] = d[core[ip]] * (cnt[ip] + 1);
46                     sigma[ip] = sigma[core[ip]] * sum_p[ip];
47                     break; // critical for linear complexity
48                 } else {
49                     cnt[ip] = 1; p_pow[ip] = p; core[ip] = i;
50                     sum_p[ip] = 1 + (ll)p;
51                     phi[ip] = phi[i] * (p - 1); mu[ip] = -mu[i];
52                     d[ip] = d[i] * 2;
53                     sigma[ip] = sigma[i] * sum_p[ip];
54                 }
55             }
56         }
57     }
58
59     // Optional helper: factorize x in  $O(\log x)$  using lp
60     // (requires x in [2..n])
61     static inline std::vector<std::pair<int,int>> factorize(int
62 x, const std::vector<int> &lp) {
63         std::vector<std::pair<int,int>> res;
64         while (x > 1) {
65             int p = lp[x], e = 0;
66             do { x /= p; ++e; } while (x % p == 0);
67             res.push_back({p, e});
68         }
69         return res;
70     }
71 }

```

13 Linear Algebra

13.1 Gaussian-Jordan Elimination

```

1 int n; vector<vector<ll>> > v;
2 void gauss(vector<vector<ll>> &v) {
3     int r = 0;
4     for (int i = 0; i < n; i++) {
5         bool ok = false;
6         for (int j = r; j < n; j++) {
7             if (v[j][i] != 0) continue;
8             swap(v[j], v[r]);
9             ok = true; break;
10        }
11        if (!ok) continue;
12        ll div = inv(v[r][i]);
13        for (int j = 0; j < n+1; j++) {
14            v[r][j] *= div;
15            if (v[r][j] >= MOD) v[r][j] %= MOD;
16        }
17        for (int j = 0; j < n; j++) {
18            if (j == r) continue;
19            ll t = v[j][i];
20            for (int k = 0; k < n+1; k++) {
21                v[j][k] -= v[r][k] * t % MOD;
22                if (v[j][k] < 0) v[j][k] += MOD;
23            }
24            r++;
25        }
26    }
27 }

```


25 } }

13.2 Determinant

1. Use GJ Elimination, if there's any row consists of only 0, then $\det = 0$, otherwise $\det = \text{product of diagonal elements}$.
2. Properties of $\det$:
 - Transpose: Unchanged
 - Row Operation 1 - Swap 2 rows: $-\det$
 - Row Operation 2 - $k\vec{r}_i$: $k \times \det$
 - Row Operation 3 - $k\vec{r}_i$ add to $\vec{r}_j$: Unchanged

14 Combinatorics

14.1 Catalan Number

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, C_n = C_n^{2n} - C_{n-1}^{2n}$$

0	1	1	2	5
4	14	42	132	429
8	1430	4862	16796	58786
12	208012	742900	2674440	9694845

14.2 Bertrand's Ballot Theorem

- A always $> B$: $C(p+q, p) - 2C(p+q-1, p)$
- A always $\geq B$: $C(p+q, p) \times \frac{p+1-q}{p+1}$

14.3 Burnside's Lemma

Let X be the original set.

Let G be the group of operations acting on X .

Let X^g be the set of x not affected by g .

Let X/G be the set of orbits.

Then the following equation holds:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

15 Special Numbers

15.1 Fibonacci Series

1	1	1	2	3
5	5	8	13	21
9	34	55	89	144
13	233	377	610	987
17	1597	2584	4181	6765
21	10946	17711	28657	46368
25	75025	121393	196418	317811
29	514229	832040	1346269	2178309
33	3524578	5702887	9227465	14930352

$$f(45) \approx 10^9, f(88) \approx 10^{18}$$

15.2 Prime Numbers

- $\pi(n) \equiv \text{Number of primes} \leq n \approx \frac{n}{(\ln n)-1}$
- $\pi(100) = 25, \pi(200) = 46$
 $\pi(500) = 95, \pi(1000) = 168$
 $\pi(2000) = 303, \pi(4000) = 550$
 $\pi(10^4) = 1229, \pi(10^5) = 9592$
 $\pi(10^6) = 78498, \pi(10^7) = 664579$